SIMATS ENGINEERING

DSA0308 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT 4 – CO4

SEMANTIC ANALYSIS AND SEMANTIC SYSTEMS ENGINEERING

STUDENT DETAILS

Particulars	**Details**
Student Name	**K. Dhanush Reddy**
Register Number	**192324087**
Department / Section	**AI & DS**
Faculty Name	**Saravanan M.S.**
Nature of Assignment	**Individual**
Duration	**1 Week**

PART I – SEMANTIC ANALYSIS THROUGH STRUCTURED REASONING

Purpose, scope and working assumptions

Part I is a design-and-analysis exercise. The system receives natural-language sentences and must convert them into representations that preserve entities, events, semantic roles, time, quantification and word senses. The source assignment explicitly states that source code is not required. Accordingly, this part uses structured tables, predicate logic, event frames, semantic graphs and ordered procedures rather than implementation code.

The six tasks follow the progression in the assignment: identify meaning components; formalise statements in first-order predicate calculus; map syntax to semantics; distinguish lexemes and senses; design meaning-based retrieval; and integrate all of these operations in a support-system challenge. We use the exact examples from Part I, including "Maya gave Ravi a book yesterday", the university predicates, the telescope attachment sentence, the three bank contexts, the library-search query and the customer-support sentence.

Our general design rule is to keep the surface sentence and the semantic representation separate. The sentence is a sequence of words with grammatical structure. The representation is a set of typed entities, events, roles, relations and constraints. If a sentence has more than one reasonable interpretation, we preserve both candidates until context or a validation rule supports one. We do not silently select a sense merely because one keyword is present.

All invented names used as constants are local identifiers for the worked examples. They do not assert facts about real people. In predicate logic, every predicate and constant is defined before the formula in which it appears. For retrieval calculations, the document collection and relevance labels are stated explicitly so that precision and recall can be checked by hand.

PART I – TASK 1: MEANING REPRESENTATION AND LANGUAGE STRUCTURE

Scenario

A conversational assistant receives "Maya gave Ravi a book yesterday." The system must represent who performed the action, what was transferred, who received it and when the event occurred. The sentence is a compact example of a transfer event with three participants or semantic roles and one temporal modifier.

Guiding enquiry

The central question is how the system can preserve the intended meaning, expose assumptions, and produce a result that can be checked. We therefore separate the input sentence, the semantic representation, the reasoning procedure and the evidence used to evaluate the result.

Task 1.1 – Identify Meaning Components

The main event is a giving or transfer event. We represent the event token as e1. The entities are Maya, Ravi and a book. Maya is the Agent or Giver because she initiates the transfer. Ravi is the Recipient because the transfer is directed to him. The book is the Theme because it is the object transferred. The temporal expression “yesterday” locates the event relative to the utterance time. The exact calendar date cannot be recovered unless the system also knows the date and time zone of the conversation.

The linguistic form is a past-tense declarative clause. Its surface order is Subject–Verb–Indirect Object–Direct Object–Temporal Adjunct. The semantic content is not simply the sequence “Maya / gave / Ravi / book / yesterday”. It is a structured event with role-labelled links. A different word order can express the same roles, so the system should not assume that the first noun is always an Agent or that the second noun is always a Recipient.

Some information remains implicit. The sentence does not say whether Ravi accepted the book, whether the transfer was physical or digital, where it happened, or whether the book was new. A safe representation should not add these facts. It can record that the verb “give” conventionally describes a transfer, but it should not infer acceptance or ownership change as a confirmed fact without a domain rule.

Table 1.1. Semantic inventory for the Part I Task 1 sentence

Unit	Value	Semantic function	Evidence in sentence
Event	e1: GIVE	Transfer event	Past-tense verb “gave”
Agent	Maya	Initiates the event	Subject noun phrase
Recipient	Ravi	Intended receiver	Indirect object
Theme	book-1	Transferred entity	Direct object
Time	yesterday	Temporal location	Temporal adverb
Tense	past	Locates event before speech time	Verb morphology

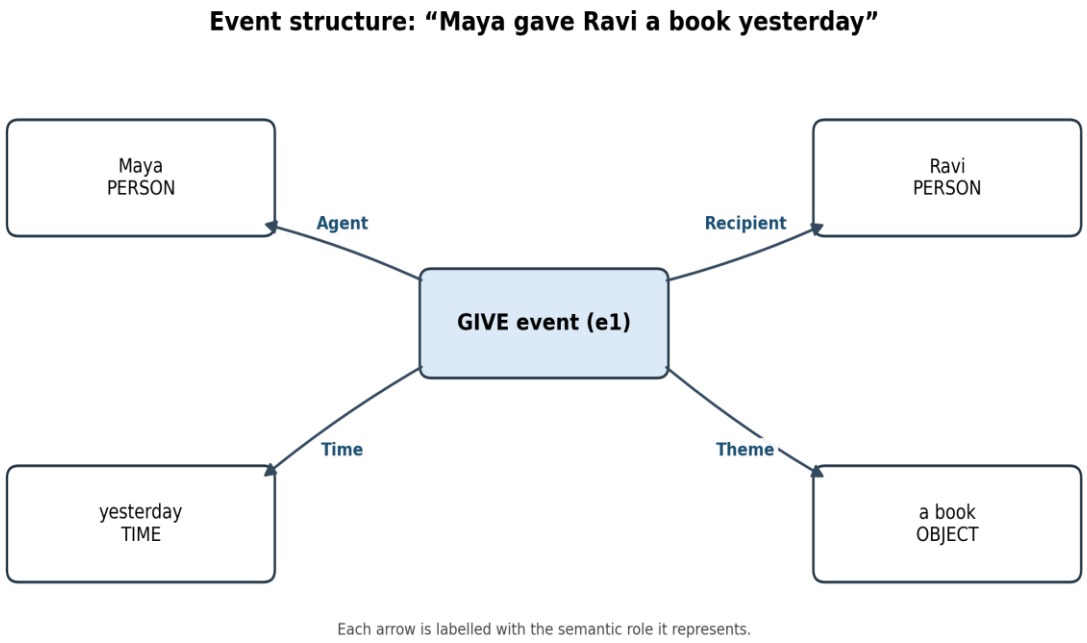


Figure 1.1. Event graph for the giving sentence.

Figure explanation: The event is the central node. Directed labelled relations connect it to the giver, recipient, theme and time. This graph makes the semantic roles explicit even if the sentence is later paraphrased.

Task 1.2 – Compare Meaning Representations

A simple conceptual/event representation can be written as GIVE(e1) with Agent(e1, Maya), Recipient(e1, Ravi), Theme(e1, book1) and Time(e1, yesterday). The event identifier keeps all roles tied to the same occurrence. This is important because a sentence can contain more than one event, and role facts must not accidentally be attached to the wrong event. The representation is compact and supports queries such as “Who received the book?” by retrieving the Recipient role of e1.

A structured frame stores the same information in named slots. The frame has a frame type GIVING and a frame identifier e1. Its slots are Agent=Maya, Recipient=Ravi, Theme=book1, Time=yesterday and Tense=past. A frame is convenient for downstream software because each slot has a predictable field name. A semantic graph is more flexible when relations connect multiple events or when an entity participates in several events. Both forms can be converted into one another if the event identifier and role labels are retained.

The event representation makes the event and its role relations explicit. The frame makes the expected fields explicit and makes missing values easy to detect. The event notation leaves details such as the internal properties of the book implicit. The frame also leaves implicit whether the event succeeded or whether Ravi accepted the book. We should add a separate Result or Acceptance slot only if the input or a trusted knowledge source supports it.

Table 1.2. Frame representation

Frame field	Assigned value	Interpretation
Frame type	GIVING	Transfer event
Event ID	e1	Unique event instance
Agent	Maya	Person who gives
Recipient	Ravi	Intended receiver
Theme	book1	Transferred object
Time	yesterday	Relative time expression
Tense	past	Grammatical tense

Task 1.3 – Meaning Structure and Ambiguity

Word order provides grammatical clues, but semantic roles are determined by the verb frame and syntactic relations together. In the original sentence, the subject is Maya and the verb "gave" licenses a recipient and a theme. The indirect object Ravi is interpreted as the Recipient, while "a book" is the Theme. The temporal adverb modifies the event rather than the book or Ravi.

A meaning-preserving alternative is "Yesterday, a book was given to Ravi by Maya." The passive construction changes the grammatical subject: "a book" is now the surface subject. The Agent remains Maya because the by-phrase identifies the giver. The Recipient remains Ravi because the prepositional phrase "to Ravi" marks the destination of the transfer. The time remains yesterday. The event representation therefore stays GIVE(e1), Agent(e1,Maya), Recipient(e1,Ravi), Theme(e1,book1), Time(e1,yesterday).

A meaning-changing alternative would be "Maya gave Ravi's book yesterday." The possessive form now indicates that the book belongs to Ravi; it does not state that Ravi received it. A system that treats any nearby name as a recipient would produce a false role assignment. We therefore use the verb's argument structure and the syntactic dependency labels rather than proximity alone.

Table 1.3. Meaning-preserving and meaning-changing variations

Sentence	Surface change	Role interpretation	Same core event?
Maya gave Ravi a book yesterday.	Active voice	Maya=Agent; Ravi=Recipient; book=Theme	Yes
Yesterday, a book was given to Ravi by Maya.	Passive voice and fronted time	Maya=Agent; Ravi=Recipient; book=Theme	Yes
Maya gave Ravi's book yesterday.	Possessive NP	Maya=Agent; book belongs to Ravi; recipient unstated	No

Task 1.4 – Design Defence

Our event/frame representation is suitable for a computer system because it assigns stable identifiers and typed roles to the information in the sentence. A program can retrieve the

recipient, compare event times, detect a missing theme or convert the frame into a database record. Unlike a paraphrase, the representation separates participants from their grammatical positions and does not depend on one fixed word order. It also records uncertainty and omitted information without inventing facts.

Real-time application example

Voice assistants use slot filling when a user says “Send the file to Arun tomorrow.” The system identifies an action, an object, a recipient and a time. The same event-frame idea supports calendar commands, package tracking and email composition. If the recipient slot is missing, the assistant can ask a clarification question rather than guessing.

Additional design analysis and boundary cases

The same design can be extended to multiple events. In “Maya gave Ravi a book and thanked him yesterday”, the system should create GIVE(e1) and THANK(e2), then represent the shared temporal modifier according to its scope. If the modifier applies to both events, it can be linked to each event or to a coordination group. If it applies only to the second event, the representation must preserve that narrower scope. This shows why stable event identifiers are more reliable than a flat paraphrase.

The event frame should distinguish required roles from optional modifiers. For a transfer event, Agent and Theme are normally required. Recipient may be optional in a sentence such as “Maya gave a book away”, depending on the chosen verb frame. Time is also optional. A validation rule must therefore be tied to the predicate schema rather than requiring every possible slot for every event. Missing optional information should be represented as unknown, not as a negative fact. This distinction matters in databases because “recipient unknown” is different from “there was no recipient”.

A useful implementation should also record the source span from which each semantic slot was extracted. For example, the Agent value Maya is supported by the subject phrase, the Theme value book1 is supported by the direct object, and the time value yesterday is supported by the temporal adverb. A source-span link allows a reviewer to inspect the original text when a slot appears incorrect. It also supports correction: if the system later discovers that the book was actually a document attachment, the Theme identifier can be updated without rewriting the event structure.

Boundary test and expected system behaviour

A further boundary test is to remove one participant at a time. “Maya gave a book yesterday” has no explicit Recipient. The system should create a GIVE event with Agent and Theme, while Recipient remains unknown. It should not insert Ravi from a previous unrelated sentence unless coreference and discourse context support that link. “Ravi received a book from Maya yesterday” changes the syntactic roles but should map to the same transfer participants. These tests check both robustness to omitted arguments and invariance under paraphrase. In a real deployment, the event schema should preserve provenance, confidence and missing-value status for every slot.

Acceptance criteria: Acceptance criteria for this representation are: every explicit participant is assigned once to the correct role; the time expression is attached to the event; the event identifier is stable; and no unsupported result such as successful receipt is added. The active and passive forms must produce equivalent role structures. A missing optional slot must remain unknown, while a missing required slot must trigger a validation warning. These criteria can be checked manually against the sentence and automatically against the event-frame schema.

Evidence Expected – how this section meets it

The entity and event inventory, role table, event graph, frame slots, meaning-preserving passive alternative and short design defence collectively provide the evidence expected for Task 1. The representations also state what remains implicit, which prevents unsupported inferences.

PART I – TASK 2: FIRST-ORDER PREDICATE CALCULUS

Scenario

A university information system stores facts about students, courses and instructors. It must express general rules about enrolment and teaching, represent existence claims, and distinguish statements whose quantifiers appear in different orders.

Guiding enquiry

The central question is how the system can preserve the intended meaning, expose assumptions, and produce a result that can be checked. We therefore separate the input sentence, the semantic representation, the reasoning procedure and the evidence used to evaluate the result.

Task 2.1 – Define the Logical Vocabulary

We use a one-sorted domain containing people and courses, with unary type predicates separating the categories. The constants are nlp for the course Natural Language Processing and rao for Dr. Rao. Variables x and y range over domain objects; the type predicates restrict their intended use. We use the conventional equality symbol = only for identity, not for similarity.

Student(x) means x is a student. Course(c) means c is a course. Instructor(i) means i is an instructor. Enrolled(x,c) means learner or student x is enrolled in course c. Teaches(i,c) means instructor i teaches course c. Passed(x,c) means student x passed course c. The argument order is fixed: the person or instructor comes first and the course comes second. This consistency is essential because Enrolled(x,c) and Enrolled(c,x) would be different and ill-typed claims.

The constant rao denotes Dr. Rao and nlp denotes the NLP course. We assume Student and Instructor are person categories, and Course is a course category. We do not assume that every instructor is a student or that every course has exactly one instructor. Those facts would need separate axioms.

Table 2.1. Predicate and constant definitions

Symbol	Type	Argument order / value	Meaning
Student(x)	Predicate	one person	x is a student
Course(c)	Predicate	one course	c is a course
Instructor(i)	Predicate	one person	i is an instructor
Enrolled(x,c)	Predicate	student, course	x is enrolled in c
Teaches(i,c)	Predicate	instructor, course	i teaches c
Passed(x,c)	Predicate	student, course	x passed c
rao	Constant	person	Dr. Rao
nlp	Constant	course	Natural Language Processing

Task 2.2 – Translate Natural Language into Logic

(i) “Every student is enrolled in at least one course.” For every object x, if x is a student, there exists a course c such that x is enrolled in c. The formula is: $\forall x$ (Student(x) $\rightarrow \exists c$ (Course(c) $\wedge$ Enrolled(x,c))). The existential quantifier is inside the scope of the universal quantifier. This means each student may have a different enrolled course; it does not require one common course for all students.

(ii) “Some student passed every course taught by Dr. Rao.” The formula is: $\exists x$ (Student(x) $\wedge$ $\forall c$ ((Course(c) $\wedge$ Teaches(rao,c)) $\rightarrow$ Passed(x,c))). The same student x must pass every course satisfying both Course(c) and Teaches(rao,c). The existential quantifier is outside the universal quantifier, so we first choose one student and then require that student to pass all Rao-taught courses.

(iii) “No instructor teaches every course.” The direct formula is: $\forall i$ (Instructor(i) $\rightarrow \neg \forall c$ (Course(c) $\rightarrow$ Teaches(i,c))). An equivalent form is $\forall i$ (Instructor(i) $\rightarrow \exists c$ (Course(c) $\wedge$ $\neg$Teaches(i,c))). The second form is often easier to read: every instructor has at least one course that they do not teach. The formula does not say that instructors teach no courses; it only denies that any instructor teaches all courses.

Table 2.2. Natural-language statements and FOPC translations

Statement	FOPC formula	Key scope point

Every student is enrolled in at least one course.	∀x(Student(x) → ∃c(Course(c) ∧ Enrolled(x,c)))	A course may differ for each student.
Some student passed every course taught by Dr. Rao.	∃x(Student(x) ∧ ∀c((Course(c) ∧ Teaches(rao,c)) → Passed(x,c)))	One selected student must pass all Rao-taught courses.
No instructor teaches every course.	∀i(Instructor(i) → ∃c(Course(c) ∧ ¬Teaches(i,c)))	Each instructor has at least one course not taught by them.

Task 2.3 – Interpret and Check the Formulae

Consider ∀x(Student(x) → ∃c(Course(c) ∧ Enrolled(x,c))). The outer quantifier ∀x ranges over all objects. The implication makes the claim relevant only when x is a student. For each student, the inner existential quantifier searches for at least one course in which that student is enrolled. The course variable c is inside the scope of ∀x, so the witness course can vary with x.

To show that quantifier order matters, compare ∀x∃y R(x,y) with ∃y∀x R(x,y). Let the domain be {a,b} and let R be true only for the pairs (a,1) and (b,2). Then ∀x∃y R(x,y) is true: a has witness 1 and b has witness 2. But ∃y∀x R(x,y) is false because there is no single y related to both a and b. The first formula permits different witnesses; the second requires one shared witness.

This finite model is a counterexample to equivalence. It is not enough to say that both formulas contain one universal and one existential quantifier. Their order changes which choices may depend on earlier variables. In a database, the distinction could separate “every student has some advisor” from “there is one advisor shared by every student”.

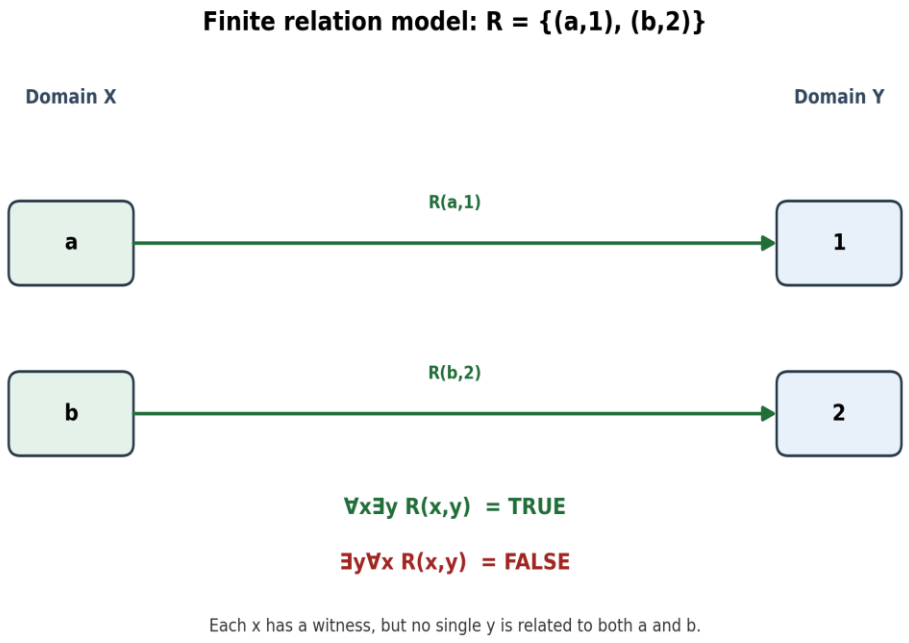


Figure 2.1. Finite relation model illustrating quantifier order.

Figure explanation: The two arrows show that each x has a y witness, but the witnesses differ. There is no single y connected to both x values. Thus the first quantifier order is true and the reversed order is false.

Table 2.3. Truth values in the finite model

x	Available y values	R(x,y) true?	Witness for ∃y
a	1, 2	R(a,1) only	1
b	1, 2	R(b,2) only	2

Task 2.4 – Logical Consistency

Let Student(x) have its earlier meaning, and let Passed(x,nlp) mean x passed NLP. “Every student passed NLP” is ∀x(Student(x) → Passed(x,nlp)). “There is a student who did not pass NLP” is ∃x(Student(x) ∧ ¬Passed(x,nlp)). These two statements cannot both be true in a classical first-order model if the domain and interpretation of Student and Passed are the same.

The existential statement supplies a particular student a for which Student(a) is true and Passed(a,nlp) is false. The universal statement requires that every student, including a, passed NLP. This creates a direct contradiction: Passed(a,nlp) and ¬Passed(a,nlp). If the statements came from different dates or different student populations, the database would need time or cohort parameters. Without such a qualification, they are inconsistent.

Real-time application example

University registration systems use predicate logic to check rules such as “every enrolled learner must satisfy a prerequisite” and “no learner can complete a course without enrolment”. A rule engine can detect missing prerequisite records and explain which learner-course pair violates a rule.

Additional design analysis and boundary cases

Quantifier scope becomes especially important when a system answers natural-language questions. “Every learner has completed a prerequisite” permits a different prerequisite for each learner. “There is a prerequisite completed by every learner” requires one common course. A user may not notice this difference in a quick paraphrase, but a registration policy can change substantially. The finite model provides a counterexample that can be checked manually and can also serve as a regression test whenever the logical translation component is modified.

Type predicates also protect the knowledge base. A formula such as Enrolled(nlp,ana) is syntactically a predicate application, but it violates the intended argument types. A typed interface should reject the course in the learner position before inference. Domain and range constraints can be stated as axioms, but an application can also enforce them when records are inserted. Consistent argument order is important when predicates are used across multiple tasks or modules.

A practical knowledge base should declare whether it uses an open-world or closed-world assumption. Under an open-world assumption, a missing fact is unknown. If Completed(ana,nlp) is not stored, the system cannot automatically conclude that Ana did not complete NLP. Under a closed-world assumption, a database application may treat missing records as false for a specific operational query, but this is a policy choice rather than a logical truth. The consistency check in this task uses classical first-order logic and distinguishes contradiction from absence of evidence.

Boundary test and expected system behaviour

A second logical test uses an empty or restricted course set. If no courses are offered, the formula $\forall c((\text{Course}(c) \wedge \text{Offered}(c)) \rightarrow \text{Completed}(x,c))$ is vacuously true for any learner x. This is a standard property of universal quantification, but it may surprise a user who expects "completed every offered course" to imply that at least one course exists. If the application requires at least one offered course, it should add $\exists c(\text{Course}(c) \wedge \text{Offered}(c))$ as a separate condition. This illustrates why formalisation must reflect policy assumptions instead of relying on informal expectations.

Acceptance criteria: Acceptance criteria for the logical component are: all predicates have documented argument order; every constant has a declared type; quantifier scope matches the English wording; the knowledge base can produce the expected inference; and the finite countermodel separates the two quantifier orders. The translator should reject ill-typed predicate arguments and should preserve the difference between false and not derivable under the selected database assumption.

Evidence Expected – how this section meets it

The vocabulary table defines every predicate and constant before use. The three translations show quantifier scope, the finite two-witness model demonstrates non-equivalence under quantifier reversal, and the final pair is checked by a direct contradiction.

PART I – TASK 3: LINGUISTIC CONCEPTS AND SYNTAX-DRIVEN SEMANTIC ANALYSIS

Scenario

The system must interpret “The experienced researcher saw the student with the telescope.” The words identify a seeing event, two human participants, an adjective modifying the researcher and a prepositional phrase that can attach to more than one constituent.

Guiding enquiry

The central question is how the system can preserve the intended meaning, expose assumptions, and produce a result that can be checked. We therefore separate the input sentence, the semantic representation, the reasoning procedure and the evidence used to evaluate the result.

Task 3.1 – Identify Linguistically Relevant Concepts

The entities are researcher-1 and student-1. The main predicate is SEE(e1). The researcher is the Experiencer or Perceiver and the student is the Perceived entity or Theme. “Experienced” is an adjectival modifier of researcher-1. “With the telescope” introduces a relation involving a telescope, but the exact semantic role depends on its syntactic attachment. If it modifies the noun phrase “the student”, it describes a property or accompaniment associated with the student. If it modifies the verb phrase “saw the student”, it can describe the instrument used by the researcher.

The preposition “with” is relational and context-sensitive. It can express possession, accompaniment, instrument or association. We should not map it to one fixed relation without considering the phrase it modifies and the event semantics. The noun “telescope” denotes an instrument capable of supporting visual observation, which makes the VP-instrument reading plausible. The NP reading remains grammatically possible: the student may be carrying or holding a telescope.

The semantic inventory therefore includes entity types PERSON and INSTRUMENT, the predicate SEE, the role relations Experiencer and Perceived, the adjective Experienced, and a candidate WITH relation. The system should preserve attachment information because it determines whether the telescope is linked to the student or to the seeing event.

Table 3.1. Concept inventory

Concept	Type	Possible relation	Interpretive contribution
researcher-1	PERSON	Experiencer(e1,researcher-1)	Person who perceives
student-1	PERSON	Perceived(e1,student-1)	Entity perceived
telescope-1	INSTRUMENT	With(student-1,telescope-1) or Instrument(e1,telescope-1)	Ambiguous PP participant
experienced	PROPERTY	Experienced(researcher-1)	Modifier of

			researcher
e1	SEE event	SEE(e1)	Main event

Task 3.2 – Syntax-to-Semantics Mapping

Analysis A attaches the PP to the noun phrase “the student”. The structure is [S [NP the experienced researcher] [VP saw [NP the student [PP with the telescope]]]]. Semantic composition first builds the NP student-1, then adds a relation between student-1 and telescope-1. The VP then combines the researcher as Experiencer and the modified student NP as Perceived. One possible representation is SEE(e1) ∧ Experiencer(e1,researcher1) ∧ Perceived(e1,student1) ∧ HasOrCarries(student1,telescope1). The exact relation should remain underspecified if the sentence does not tell us whether the student holds, owns or is accompanied by the telescope.

Analysis B attaches the PP to the verb phrase. Its structure is [S [NP the experienced researcher] [VP [VP saw the student] [PP with the telescope]]]. The PP contributes an instrument relation to the seeing event. The representation is SEE(e1) ∧ Experiencer(e1,researcher1) ∧ Perceived(e1,student1) ∧ Instrument(e1,telescope1). This reading says the researcher used the telescope to see the student.

The two interpretations diverge when the PP is combined. Before that point, the event predicate, the researcher and the student are shared. The NP attachment adds a relation whose first argument is student1. The VP attachment adds a relation whose first argument is event e1. This difference in argument type is a useful validation clue: Instrument expects an event, while HasOrCarries expects an entity.

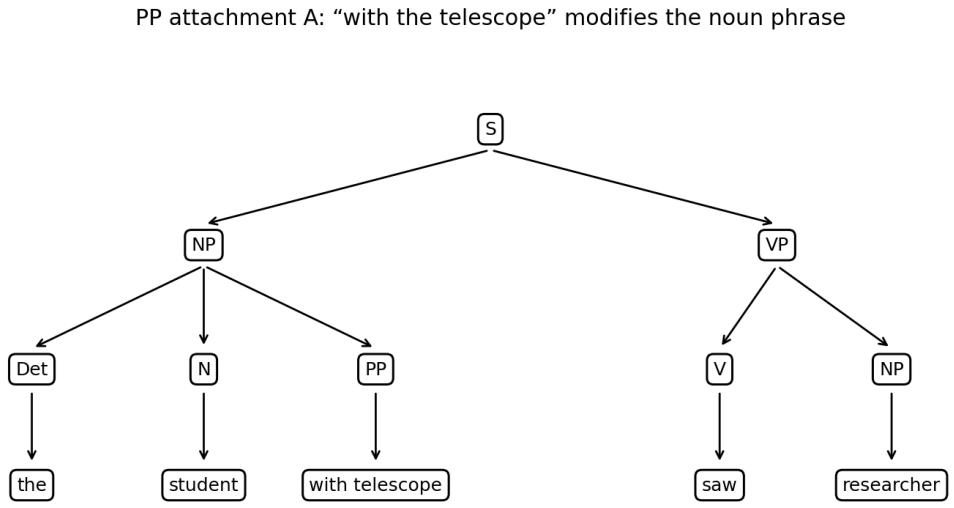


Figure 3.1. NP attachment: the PP modifies “the student”.

Figure explanation: The PP is a child of the object noun phrase. It contributes information about the student rather than about the act of seeing. The interpretation links the student to the telescope.

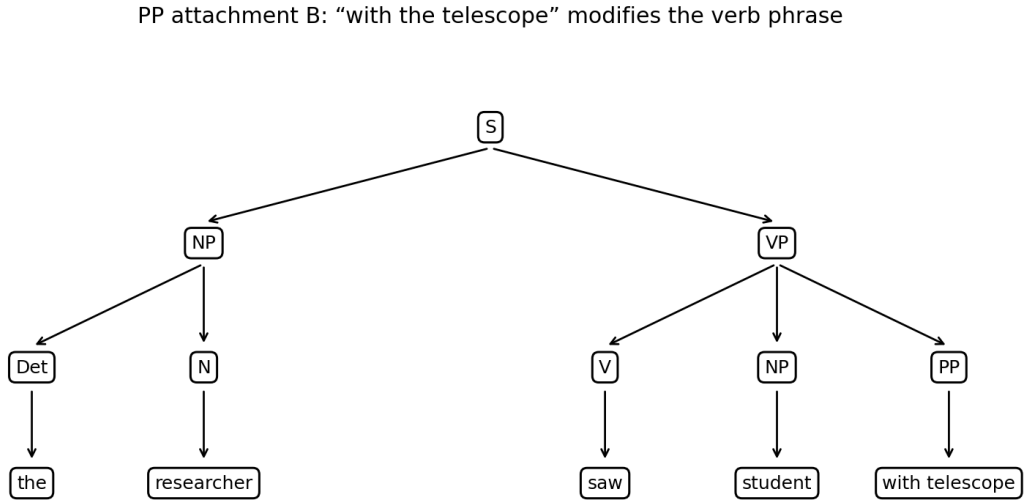


Figure 3.2. VP attachment: the PP modifies “saw the student”.

Figure explanation: The PP is attached to the verb phrase. It contributes an instrument or manner relation to the seeing event. The telescope is therefore associated with the researcher’s act of seeing.

Table 3.2. Competing semantic analyses

Analysis	Attachment	Semantic representation	Plain-language reading
A	NP	SEE(e1) ∧ Experiencer(e1,r1) ∧ Perceived(e1,s1) ∧ HasOrCarries(s1,t1)	The student has or carries the telescope.
B	VP	SEE(e1) ∧ Experiencer(e1,r1) ∧ Perceived(e1,s1) ∧ Instrument(e1,t1)	The researcher used the telescope to see the student.

Task 3.3 – Semantic Composition

Step 1: lexical lookup assigns the noun “researcher” the type PERSON and “student” the type PERSON. “Telescope” is typed as INSTRUMENT. The verb “saw” is mapped to a past-tense form of SEE with two core roles: Experiencer and Perceived. Step 2: the determiner “the” introduces a contextually identifiable entity rather than a new descriptive property. Step 3: “experienced” contributes a property to researcher-1. Step 4: the verb phrase combines SEE with its subject and object roles.

Step 5: the parser evaluates the PP attachment alternatives. Under NP attachment, the PP is composed with the object NP and introduces a relation involving student-1. Under VP attachment, the PP is composed with the event predicate and introduces an Instrument relation. Step 6: semantic type checks verify that the relation arguments have appropriate

types. Step 7: discourse context can rank the alternatives, but the parser should retain both if no evidence is decisive.

The composition is syntax-driven because the parse tree determines the constituent to which the PP contributes. A bag-of-words representation cannot distinguish the two structures: both readings contain the same words. The syntactic attachment controls the scope and argument position of the relation.

Task 3.4 – Failure Analysis

Error 1 is a wrong attachment. If the parser chooses NP attachment when the researcher actually used a telescope, the system may report that the student possesses the instrument. A validation check can compare the semantic role with selectional preferences and discourse context. If the user later says "I adjusted the focus before observing", that context strongly supports the instrument reading. The system should retain uncertainty when both readings remain plausible.

Error 2 is assigning semantic roles from word order alone. A passive paraphrase such as "The student was seen by the experienced researcher" changes the grammatical subject while preserving the Perceived and Experiencer roles. A system that labels the first noun as Agent or Experiencer without using the predicate and syntactic dependencies may reverse the roles. The validation check is to verify that the semantic role assignment agrees with the verb frame and dependency relations, including passive voice.

An additional risk is overcommitting the relation "with" to possession. The PP does not prove ownership. We therefore use an underspecified relation such as WITH-ASSOCIATED until the context identifies possession, accompaniment or instrument. This is safer than storing a stronger claim that the sentence does not entail.

Real-time application example

Search assistants and document question-answering systems face the same PP-attachment issue. A legal query such as "the witness with the recording" may identify which witness is meant, while "recorded the witness with the device" identifies an instrument. Correct attachment changes which entity or event a search filter should use.

Additional design analysis and boundary cases

Selectional preferences should be treated as ranking evidence, not as hard truth. A telescope is a plausible instrument for seeing, but a student can also carry one. A camera is a plausible instrument for noticing in a surveillance setting, but a student can carry a camera. If the system makes the most likely interpretation a hard commitment too early, later context cannot

repair it. A packed representation of alternatives is therefore useful until the dialogue provides enough evidence.

The location phrase near the library creates a second attachment decision. It can describe the student's location or the location of the noticing event. A parser that resolves the camera phrase but ignores the location phrase has not fully represented the sentence. The system should keep an attachment record for each PP, including its head, semantic relation and confidence. A user question such as "Where was the student?" can then select the entity-level location, while "Where did the analyst notice the student?" selects the event-level location.

The two PP attachments are not simply two word-order variants. They assign different semantic types to the relation introduced by with. In the NP reading, the first argument is a person or object. In the VP reading, the first argument is an event. A semantic type system can reject a relation application that uses an event where an entity is expected, or an entity where an event is required. It cannot always decide which well-typed reading the speaker intended, but it can eliminate malformed combinations.

Boundary test and expected system behaviour

A useful evaluation pair is "The analyst noticed the student with the camera" and "The analyst noticed the student who had the camera". The relative clause makes the student-level relation more explicit. Another pair is "Using the camera, the analyst noticed the student", which strongly favours the event-instrument reading. If the parser produces the same attachment distribution for all three sentences, its syntax-to-semantics mapping is not sensitive enough to the structure. These controlled contrasts are suitable for a small manual evaluation set.

Acceptance criteria: Acceptance criteria for syntax-to-semantics mapping are: both PP attachments are represented; the camera relation has the correct argument type; the Experiencer and Perceived roles are not reversed; and the location phrase is not silently discarded. The parser should retain more than one interpretation when the input provides no decisive evidence. Any hard selection should have a traceable context or type-based reason.

Evidence Expected – how this section meets it

The concept inventory identifies the entities, event, modifier and relation. Two parse structures are shown with corresponding semantic representations. The composition trace identifies the exact divergence point, and the failure analysis gives a validation check for attachment errors and role reversal.

PART I – TASK 4: LEXEMES, SENSES AND WORD SENSE DISAMBIGUATION

Scenario

The lexeme BANK appears in three contexts: “Nila deposited cash at the bank”, “They sat on the bank of the river” and “The pilot banked the aircraft to the left”. The task is to distinguish the underlying lexeme, its word forms, its grammatical categories and the intended senses.

Guiding enquiry

The central question is how the system can preserve the intended meaning, expose assumptions, and produce a result that can be checked. We therefore separate the input sentence, the semantic representation, the reasoning procedure and the evidence used to evaluate the result.

Task 4.1 – Lexemes and Senses

A lexeme is an abstract vocabulary item that groups related word forms. A word form is the surface form that appears in a sentence. A sense is one contextually selected meaning of a lexeme. In the first sentence, bank is a noun meaning a financial institution where money can be deposited. In the second sentence, bank is a noun meaning the sloping land beside a river. In the third sentence, banked is a past-tense verb form of the lexeme BANK, meaning to tilt or incline an aircraft during a turn.

The three contexts therefore show two noun senses and one verb sense. The spelling bank is shared by the noun and base verb, while banked is an inflected verb form. Part-of-speech information is an important first filter: “at the bank” occurs after a determiner and preposition and behaves as a noun; “the bank of the river” is also a noun; “the pilot banked” is a finite verb construction. A sense inventory must preserve both grammatical category and meaning.

We assign sense identifiers BANK_N_FIN, BANK_N_RIVER and BANK_V_TILT. These identifiers are local labels for this assignment. The inventory does not imply that these are the only senses in English. It is a compact domain inventory for the three required examples.

Table 4.1. Lexeme and sense inventory

Sense ID	Part of speech	Definition	Contextual cues
BANK_N_FIN	Noun	Financial institution	deposited, cash, account, loan, teller
BANK_N_RIVER	Noun	Land along the edge of a river	river, water, shore, sat, slope
BANK_V_TILT	Verb	Tilt an aircraft while turning	pilot, aircraft, left, turn, wings

Task 4.2 – Design a Word Sense Disambiguation Method

We design a deterministic, context-based baseline. Input consists of a tokenised sentence, the target occurrence of bank or banked, and a part-of-speech tag. The output is a sense identifier, a confidence score and a short list of matched features. The method uses a window of up to

four content words on either side of the target, excluding punctuation and common function words. It also examines the local dependency relations, such as the noun modified by “river” or the subject of the verb “banked”.

Candidate generation first uses part of speech. A noun target receives BANK_N_FIN and BANK_N_RIVER. A verb target receives BANK_V_TILT. The feature dictionary contains financial cues (deposit, cash, money, account, loan, teller, withdraw), river cues (river, water, shore, edge, stream, sat), and aviation cues (pilot, aircraft, plane, wing, turn, left, right). Each matched cue contributes a weight of 2. A dependency cue contributes an additional weight of 1 when it appears in a compatible relation. The sense with the highest score is selected only if its score is at least 2 and exceeds the next candidate by at least 1. Otherwise the system returns “unresolved” and keeps the candidate list.

This rule is reproducible because the feature lists, window and threshold are fixed. It is not a complete language model. It may fail on metaphor, unfamiliar vocabulary, negation or contexts where the cue words are misleading. The uncertainty output is part of the design, not an error to be hidden.

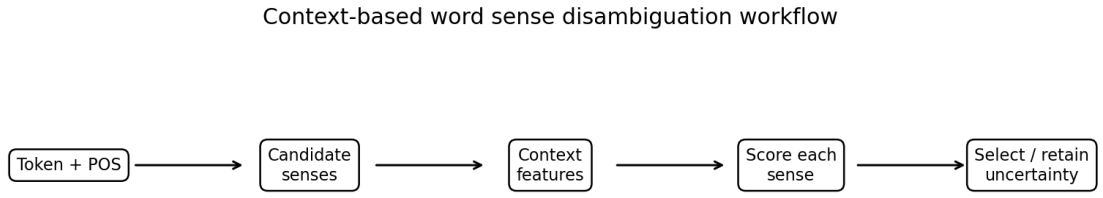


Figure 4.1. Context-based WSD decision flow.

Figure explanation: The system first identifies the target token and its grammatical category, then generates candidate senses, extracts local context features and scores each candidate. A low-margin result remains unresolved instead of forcing a possibly incorrect sense.

Table 4.2. WSD decision rules

Stage	Operation	Decision or output
Input	Locate target and POS	Target token, sentence, POS
Candidate generation	Filter by POS	Two noun senses or one verb sense
Feature extraction	Collect window and dependency cues	Financial, river or aviation features
Scoring	Add 2 per cue and 1 for compatible dependency	One score per candidate
Selection	Require top score ≥ 2 and margin ≥ 1	Sense ID or unresolved
Output	Store explanation	Sense, score, matched cues

Task 4.3 – Test the Method

Example A: “Nila deposited cash at the bank.” The target is a noun. The context window contains deposited and cash. Both are financial cues, so BANK_N_FIN receives 4 points and BANK_N_RIVER receives 0. The output is BANK_N_FIN. The explanation is direct: deposited cash is a common transaction at a financial institution.

Example B: “They sat on the bank of the river.” The target is a noun. The cues river and sat are present; river is a strong domain cue for BANK_N_RIVER. The method returns BANK_N_RIVER with a score of at least 2. The relation “of the river” also supports the geographical reading. Example C: “The pilot banked the aircraft to the left.” POS tagging identifies banked as a verb. The verb candidate is BANK_V_TILT. Pilot, aircraft and left provide three aviation cues, so the verb sense is selected.

Our new sentence is “The hikers rested on the bank beside the stream.” The target is a noun. Hikers, rested, stream and beside support the land-by-water reading. The system returns BANK_N_RIVER. A likely failure is “The bank approved a river restoration loan.” Financial and river cues appear together. The nearest cue rule may score both noun senses. A more advanced system would use dependency structure: “approved” takes bank as an institutional subject, while “river restoration loan” is the topic of the loan rather than the meaning of bank. If the scores remain close, the baseline should abstain.

Table 4.3. Worked WSD traces

Input sentence	POS	Matched evidence	Selected sense
Nila deposited cash at the bank.	Noun	deposited, cash	BANK_N_FIN
They sat on the bank of the river.	Noun	river, sat, of the river	BANK_N_RIVER
The pilot banked the aircraft to the left.	Verb	pilot, aircraft, left	BANK_V_TILT
The hikers rested on the bank beside the stream.	Noun	hikers, rested, stream	BANK_N_RIVER
The bank approved a river restoration loan.	Noun	financial and river cues compete	Unresolved / require syntax

Task 4.4 – Evaluation

We create a small labelled test set with eight instances, balanced across the three senses as far as possible. Each instance is annotated by a human reader using the sentence context and the sense definitions in Table 4.1. The gold label is fixed before the method is evaluated. We count a prediction as correct only when the predicted sense identifier exactly matches the gold label. An unresolved output counts as incorrect for the simple accuracy calculation, although it should also be reported separately as abstention.

Accuracy = number of correct predictions / total number of test instances. Suppose the method correctly labels 7 of 8 examples. Accuracy = 7/8 = 0.875 = 87.5%. This is a hand-

calculated illustrative evaluation for the designed rule set, not a result from an executed program. A fuller report should include the confusion matrix and per-sense recall because one overall number can hide a weak minority sense.

A balanced test set gives each sense similar influence. Real text may contain the financial noun far more often than the aviation verb. A system that always predicts the common financial sense could score well on an imbalanced test set while failing river and aviation contexts. We therefore report per-class precision and recall, the class distribution, and macro-averaged performance. Annotation ambiguity also matters: a sentence such as "The bank was steep" may refer to a river bank or a financial institution's physical building. The test policy should allow an ambiguous label or exclude the sentence from single-sense accuracy if context is insufficient.

Table 4.4. Eight-instance evaluation set and illustrative predictions

ID	Gold sense	Predicted sense	Correct?
1	BANK_N_FIN	BANK_N_FIN	Yes
2	BANK_N_FIN	BANK_N_FIN	Yes
3	BANK_N_FIN	BANK_N_RIVER	No
4	BANK_N_RIVER	BANK_N_RIVER	Yes
5	BANK_N_RIVER	BANK_N_RIVER	Yes
6	BANK_N_RIVER	BANK_N_RIVER	Yes
7	BANK_V_TILT	BANK_V_TILT	Yes
8	BANK_V_TILT	BANK_V_TILT	Yes

Accuracy calculation: 7 correct ÷ 8 total = 0.875 = 87.5%. The numbers in this small table are illustrative hand-calculated results for the proposed rule-based method; they are not presented as measured software output.

Real-time application example

WordNet-based WSD systems and search engines use sense inventories, lexical relations and local context to distinguish meanings. A medical search system must distinguish a word used as an anatomical term from the same spelling used in ordinary language. The same staged approach—POS, candidate senses, context features, scoring and evaluation—makes the decision auditable.

Additional design analysis and boundary cases

Accuracy is easy to understand but does not describe all failure patterns. A confusion matrix shows whether the river sense is often confused with the financial sense or whether the verb sense is missed. Macro-averaged recall gives each class equal weight, while micro-averaged accuracy gives more influence to common classes. For a helpdesk or search application, an

abstention option may be safer than a wrong confident prediction. The system should report both its accuracy on decided cases and the percentage of inputs for which it abstains.

The sense inventory must be stable during evaluation. If annotators add a new sense after seeing a system error, the gold labels and the model output are no longer directly comparable. We should write definitions and examples before annotation, then allow an “uncertain” label for contexts that genuinely do not determine one sense. Annotators should record the evidence they used, especially when a context contains cues for more than one sense. Disagreements should be adjudicated or reported rather than hidden.

A context window is a practical starting point, but the best features depend on the grammatical category of the target. For a noun, nearby determiners and prepositions help identify its syntactic role. For a verb, the subject and object may provide stronger clues than the immediately adjacent words. In “The pilot banked the aircraft to the left”, the subject pilot and object aircraft are more informative than a fixed distance window alone. A dependency parser can expose these relations and reduce errors caused by unrelated words in the sentence.

Boundary test and expected system behaviour

The evaluation should include both clear and ambiguous examples. Clear contexts include “The pilot banked the aircraft” and “Nila deposited cash at the bank”. Ambiguous contexts include “They waited by the bank” without any river or financial cues. A good WSD system should not be rewarded for guessing the majority sense in the ambiguous case. We can report accuracy on resolved examples, abstention coverage and the percentage of ambiguous items correctly retained as uncertain. This makes the evaluation closer to the actual user experience.

Acceptance criteria: Acceptance criteria for WSD are: the candidate inventory is fixed before testing; POS is used to restrict candidates; each selected sense has visible contextual evidence; unknown senses can be left unresolved; and accuracy is calculated against labels assigned independently. The evaluation report should include class counts and should not present a small constructed test as evidence of unrestricted language performance.

Evidence Expected – how this section meets it

The sense inventory distinguishes lexeme, word form, part of speech and sense. The WSD procedure specifies input, candidate generation, context window, features, scoring threshold and output. Four worked examples, a failure case and an eight-instance accuracy calculation provide the requested test and evaluation evidence.

PART I – TASK 5: INFORMATION RETRIEVAL AND SEMANTIC MATCHING

Scenario

A library search system receives the query “methods for finding the meaning of ambiguous words”. It must retrieve documents that explain word sense disambiguation, context-based interpretation and semantic methods, even when a relevant document does not repeat the query’s exact wording.

Guiding enquiry

The central question is how the system can preserve the intended meaning, expose assumptions, and produce a result that can be checked. We therefore separate the input sentence, the semantic representation, the reasoning procedure and the evidence used to evaluate the result.

Task 5.1 – Query and Document Representation

The query’s central intent is to find methods that determine which meaning of a word is intended in a particular context. The key concepts are ambiguous word, context, lexical meaning, sense selection, disambiguation procedure and evaluation. The user is not merely asking for dictionary definitions of individual words; the phrase “methods for finding” indicates an information-seeking intent about techniques or algorithms.

We represent the query as a concept vector containing WSD, word sense, context, lexical ambiguity, supervised learning, knowledge-based method and semantic interpretation. A document is represented by its title, abstract and body concepts, plus a vector of weighted terms. We keep field information because a concept in the title is often a stronger relevance signal than a single occurrence in a reference list. Synonyms and related terms are mapped to a shared concept where justified: “sense selection” and “word sense disambiguation” are related, while “dictionary lookup” alone may be too broad.

The candidate collection includes relevant documents with different wording and distractors that share a keyword. This is necessary to test the difference between literal overlap and semantic relevance. We use six documents so that the ranking and metric calculation remain easy to inspect.

Table 5.1. Query concepts and candidate document collection

Document	Short description	Relevant?	Reason
D1	A guide to word sense disambiguation using local context	Yes	Directly explains sense selection
D2	A survey of lexical ambiguity resolution with supervised and knowledge-based	Yes	Uses alternate wording but covers the same intent

	methods		
D3	A dictionary of ambiguous English words and their definitions	No	Lists meanings but does not explain methods
D4	Contextual embeddings for selecting a word's intended meaning	Yes	Explains a semantic method for resolving senses
D5	A guide to finding words in a digital library catalogue	No	Shares finding/word terms but concerns search navigation
D6	A history of English spelling and word formation	No	Language topic but not sense disambiguation

Task 5.2 – Lexical versus Semantic Retrieval

Keyword retrieval tokenises the query and scores documents by literal overlap. It is fast and transparent, but it can miss relevant documents that use different terminology. For example, D4 describes contextual embeddings that select an intended meaning. It may use "contextual representation" and "sense selection" without repeating the exact phrase "ambiguous words". A strict keyword matcher may give it a low score even though it answers the question.

Semantic retrieval maps the query and documents into concept representations or embeddings. It can identify that "lexical ambiguity resolution" is closely related to "finding the meaning of ambiguous words". It can also retrieve D4 because contextual embeddings are a method for selecting a word sense. Semantic similarity is not a guarantee of relevance: D3 may be close in topic because it lists ambiguous words, but it does not explain methods. D5 may share the words "finding" and "words" while addressing a different task.

The design therefore combines lexical evidence with semantic concept coverage and sense compatibility. Literal overlap remains useful for exact terms, names and technical phrases. Semantic matching provides recall for paraphrases. A relevance check is still needed to avoid documents that are merely related to the topic.

Task 5.3 – Ranking Design

We define three normalised signals between 0 and 1. L is lexical relevance, measured by weighted query-term overlap. C is concept similarity, measured by similarity between query and document concept vectors. S is sense or intent compatibility, a score assigned by checking whether the document explains sense selection rather than merely listing word definitions. The final score is $\text{Score}(d) = 0.30L(d) + 0.45C(d) + 0.25S(d)$. Concept similarity receives the largest weight because the query is semantic in nature. Lexical relevance still contributes 30 percent so that exact technical matches are rewarded.

Tie handling is deterministic. If two documents have equal scores to three decimal places, the system first prefers the document with higher concept coverage, then higher title-field

relevance, then the lower stable document ID. The tie rule does not change relevance labels; it only makes ranking reproducible.

The following values are a small worked example. They are illustrative scores assigned from the stated features, not the output of a running search engine. The arithmetic is shown so the ranking can be checked. D4 receives a high concept and intent score despite moderate literal overlap. D3 receives strong lexical overlap but a lower intent score because it is a dictionary list rather than a methods document.

Table 5.2. Worked ranking scores

Doc	L	C	S	Calculation	Final score
D1	0.90	0.92	0.95	0.30(.90)+0.45(.92)+0.25(.95)	0.923
D2	0.72	0.96	0.98	0.30(.72)+0.45(.96)+0.25(.98)	0.884
D3	0.88	0.58	0.35	0.30(.88)+0.45(.58)+0.25(.35)	0.603
D4	0.55	0.94	0.93	0.30(.55)+0.45(.94)+0.25(.93)	0.815
D5	0.62	0.22	0.10	0.30(.62)+0.45(.22)+0.25(.10)	0.316
D6	0.20	0.30	0.12	0.30(.20)+0.45(.30)+0.25(.12)	0.231

Illustrative retrieval score components

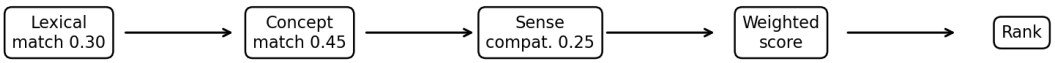


Figure 5.1. Ranking signals and combination process.

Figure explanation: Lexical overlap, concept similarity and sense compatibility are measured separately. The weighted score combines them into one ranking value; it does not treat similarity as proof that a document answers the question.

Task 5.4 – Evaluation and Error Analysis

Let the relevant set be {D1,D2,D4}. The ranking order from Table 5.2 is D1, D2, D4, D3, D5, D6. At cutoff k=4, the retrieved set is {D1,D2,D4,D3}. Three retrieved documents are relevant and one is not. Precision@4 = relevant retrieved / total retrieved = 3/4 = 0.75. Recall@4 = relevant retrieved / all relevant documents = 3/3 = 1.00.

The false positive is D3. It contains definitions of ambiguous words and may share query terms, but it does not explain methods for identifying intended meaning. The false-negative set at cutoff 2 would contain D4: if only two results were displayed, D4 would be missed even though it is relevant. At cutoff 4 there is no false negative in this small collection. A specific improvement is to add an answer-type feature that checks whether the document describes a procedure, algorithm or evaluation method. This would lower D3's intent score while preserving D4.

Precision and recall are different. Precision measures how much of the retrieved set is relevant. Recall measures how much of the full relevant set was found. A system can raise precision by returning fewer documents but then lose recall. The cutoff must therefore be stated whenever metrics are reported.

Table 5.3. Retrieval metric calculation at cutoff 4

Quantity	Count	Formula	Value
True positives	3	Relevant retrieved	3
False positives	1	Retrieved but not relevant	1
False negatives	0	Relevant but not retrieved	0
Precision@4	3 of 4	TP / (TP + FP)	3/4 = 0.75
Recall@4	3 of 3	TP / (TP + FN)	3/3 = 1.00

These scores are exact hand calculations for the illustrative six-document collection. They are not measured performance on a real library corpus.

Real-time application example

A university library search system can retrieve articles about WSD even when the user searches for “how to choose the right meaning of a word”. Google-style semantic ranking combines query understanding with document relevance signals. A legal or medical search engine can use the same principle, but must apply domain-specific terminology and stronger relevance checks.

Additional design analysis and boundary cases

A retrieval system should also consider document quality and freshness when those properties matter. A highly similar but outdated policy document may be less useful than a current official guide. Field weighting can give more importance to a title and abstract, while authority and date can be additional signals. These factors should be added only when they are relevant to the user’s request. A ranking score is a decision aid, not a proof that the top document is correct.

The relevance label should describe whether a document answers the information need, not whether it merely discusses the same topic. D3 is a false positive because it is about finding a word in a record bank, not finding the intended sense of a word. D4 is a false negative at the selected cutoff because it uses “lexical ambiguity resolution” instead of the exact query wording. These errors point to two different improvements: intent-sensitive filtering for D3 and semantic expansion for D4.

The weighted ranking formula encodes a design choice. Giving semantic coverage the largest weight improves retrieval of paraphrases, but it can also raise documents that discuss a nearby concept without answering the query. Increasing lexical weight improves exact-match precision for technical phrases but may lower recall for alternate wording. The weights should be selected using a labelled development set and then evaluated on a separate test set. They should not be adjusted on the test set until the desired score appears.

Boundary test and expected system behaviour

A ranking system should be tested with a query reformulation. “How do I determine which meaning of a word is intended?” should retrieve substantially the same relevant documents as the original query. If the ranking changes completely, the system may depend too strongly on exact wording. We can also test a deliberately broad query such as “ambiguous words”, where the user intent is less specific. In that case the system may return dictionaries and lists of examples as well as WSD methods. The ranking should reflect the narrower method-seeking intent only when the query includes that intent.

Acceptance criteria: Acceptance criteria for retrieval are: the query intent is represented explicitly; the collection contains both relevant paraphrases and keyword distractors; the ranking formula is normalised and documented; the cutoff is stated; and precision and recall use the full relevant set as the denominator. The false-positive and false-negative examples should lead to specific changes in the ranking design.

Evidence Expected – how this section meets it

The query is represented by explicit concepts, the six-document collection includes paraphrased relevant items and keyword distractors, the ranking formula uses three signals, and the worked scores produce a transparent ranking. Precision and recall are calculated at a stated cutoff with a false-positive analysis.

PART I – TASK 6: INTEGRATED SEMANTIC ANALYSIS CHALLENGE

Scenario

A customer-support system receives “I saw her duck near the bank, and it was closed.” The system must identify the ambiguities, construct candidate meanings, use syntax and context to resolve them, and retrieve a support article without matching the word bank in the wrong sense.

Guiding enquiry

The central question is how the system can preserve the intended meaning, expose assumptions, and produce a result that can be checked. We therefore separate the input sentence, the semantic representation, the reasoning procedure and the evidence used to evaluate the result.

Stage 1 – Interpret the Problem

The sentence contains at least three sources of ambiguity. First, “duck” can be a noun meaning a bird or a verb meaning to lower the head or body quickly. The sequence “her duck” can therefore be a noun phrase with her as a possessive determiner, or a verb phrase in which her is the object pronoun and duck is the verb. Second, “bank” can mean a financial institution or land beside a river. Third, the pronoun “it” may refer to the bank, the duck, or another salient entity in the discourse. The predicate “was closed” makes a financial institution a plausible antecedent, but it does not prove it.

The most useful additional information would be the user’s topic and the referent of “it”. If the user is asking about opening hours, a financial institution is likely. If the user is describing a walk beside a river, the geographical sense is likely. If the conversation concerns an animal, the noun duck reading is likely. The system should also check whether “closed” is being used literally for a building or metaphorically for an access route.

Stage 2 – Build Candidate Interpretations

Candidate A: “her duck” is a noun phrase meaning a bird belonging to or associated with a woman. The event is SEE(e1), with speaker as Experiencer and duck-1 as Perceived. The phrase “near the bank” locates the duck near a financial institution. “It” refers to the bank, and the bank was closed. A structured representation is SEE(e1) ∧ Experiencer(e1,speaker) ∧ Perceived(e1,duck1) ∧ Near(duck1,bank1) ∧ FinancialBank(bank1) ∧ Closed(bank1). The possessive relation can be represented as AssociatedWith(duck1,her-person) without assuming legal ownership.

Candidate B: “her duck” is a verb phrase meaning that the woman lowered her head or body. The speaker saw the action near a financial bank. The representation is DUCK_ACTION(e2) ∧ Agent(e2,her-person) ∧ SEE(e1) ∧ Experiencer(e1,speaker) ∧ Perceived(e1,e2) ∧ Near(e2,bank1) ∧ FinancialBank(bank1) ∧ Closed(bank1). Here “her” is an object of saw in the surface construction and the event of ducking is what the speaker perceived. The bank is again financial because “closed” is a strong context cue.

Candidate C is a weaker geographical alternative: the duck or ducking event is near a river bank. It requires “it” to refer to a contextually available object that can be closed, such as a gate, visitor centre or access path. Since no such object is named, the sentence alone does not support that referent strongly. The system should not silently insert a gate. It can retain this candidate with low confidence and ask what was closed.

Table 6.1. Candidate interpretation comparison

Candidate	Duck analysis	Bank analysis	Referent of it	Representation

				focus
A	Noun: bird	Financial institution	Bank	SEE event perceives a bird
B	Verb: lower body/head	Financial institution	Bank	SEE event perceives a ducking event
C	Noun or verb	River bank	Unstated gate/path (assumption required)	Geographical location; antecedent unresolved

Stage 3 – Syntax-Driven Analysis and WSD

The parser first produces two local structures for "saw her duck". Structure A is [VP saw [NP her [N duck]]]. Structure B is [VP saw [NP her] [VP duck]]. The first structure treats her as a determiner and duck as a noun. The second treats her as an object pronoun and duck as a bare verb. The parser then identifies the PP "near the bank" as a location modifier. It can attach to the duck NP or the ducking event depending on the chosen structure.

The WSD stage generates candidates for bank and scores context. The cue "closed" strongly supports the financial institution sense because banks have business opening hours and can be closed. The cue "near" alone does not distinguish financial from river senses. For duck, the part-of-speech and parse alternatives are decisive: a determiner-like possessive followed by a noun supports the bird reading; an object pronoun followed by a base-form verb supports the action reading. The system should use the full parse rather than a nearest-keyword rule.

If the two parses remain close, the assistant should ask a clarification question such as "Do you mean her pet duck, or that she lowered her head?" If the referent of "it" is uncertain, it can ask "Are you referring to the bank being closed?" The assistant should not return an article until the selected meaning is sufficiently stable.

Stage 4 – Information Retrieval Trace

Suppose context selects Candidate A: a bird was seen near a financial institution, and the institution was closed. The query representation contains the concepts FINANCIAL_INSTITUTION, OPENING_HOURS, CLOSED_STATUS and LOCATION_NEAR. It should not contain RIVER_BANK as the intended sense. Candidate documents include a bank branch opening-hours article, a riverbank safety article, a duck-care article and a general dictionary entry for bank. The ranking stage checks both the concept match and the sense of bank.

Document R1 explains how to check a bank branch's opening hours and holiday closure. R2 explains safety near river banks. R3 explains care for pet ducks. R4 is a dictionary page containing both financial and geographical definitions of bank. The financial branch article receives the highest intent score because it directly answers the likely support need. The riverbank article is a lexical distractor and should be rejected even though it contains the word bank.

An illustrative ranking is: R1 score 0.94, R4 score 0.61, R3 score 0.30 and R2 score 0.18. The values are illustrative and would need to be replaced by scores from an actual retrieval implementation. The ranking explanation is more important than the exact values: R1 matches the intended financial sense and the closed-status intent, while R2 matches only the surface word bank.

Table 6.2. Integrated retrieval trace

Document	Main content	Sense match	Intent match	Illustrative score
R1	Branch opening hours and closure status	Financial bank: strong	Closed status: strong	0.94
R2	Safety near a river bank	River bank: wrong sense	Low	0.18
R3	Pet duck care	Duck noun: partial	Does not answer closure	0.30
R4	Dictionary entry for bank	Contains both senses	General reference only	0.61

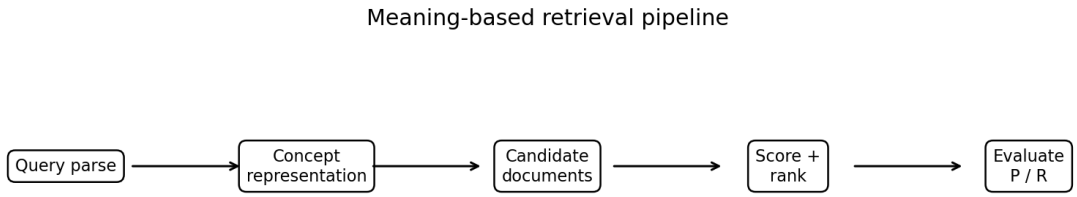


Figure 6.1. Integrated meaning-based retrieval pipeline.

Figure explanation: The selected interpretation is converted into a query representation before document matching. Sense compatibility is checked during ranking, so a document containing the word bank in the river sense does not outrank a document about financial branch closure.

Stage 5 – Challenge and Refine

A misleading context could mention a river walk immediately before the sentence, making the river-bank sense seem likely, while the phrase “it was closed” actually refers to a financial branch mentioned earlier in the conversation. A simple nearest-context system may select the river sense and retrieve a river-safety article. The design would then fail the intended support goal because the antecedent and topic are drawn from a wider discourse, not just the nearest sentence.

The refinement is to maintain a discourse entity list with recency, semantic type and salience. Pronoun resolution should test whether the antecedent is compatible with “closed” and whether it is active in the discourse. The WSD stage should combine local cues with the discourse state and keep alternative senses when the scores are close. A final answer should

expose the assumption, for example: “I interpreted ‘it’ as the bank branch. Is that what you meant?” This small confirmation prevents a confident but incorrect retrieval.

Real-time application example

Customer-support chatbots routinely resolve words such as “account”, “branch”, “card” and “charge” using the conversation topic. A banking assistant should distinguish a river bank from a financial institution and distinguish a card payment charge from an electrical charge. It should ask a clarifying question when the context does not support a single sense.

Additional design analysis and boundary cases

The best clarification question targets the uncertainty that most changes the retrieval result. Asking about duck is useful if the bird-versus-action distinction changes the article. Asking whether the bank is a branch is useful if the requested support information concerns opening hours. A good assistant should not ask the user to restate the entire message. It should expose the two plausible readings in concise language and request only the missing distinction.

A dialogue manager can maintain a small state containing entities, their types, the most recent mention, the semantic role in the current event and the confidence of each sense. The candidate bank can be scored using both lexical cues and the predicate “closed”. A financial institution is compatible with a closure status, while a river bank is not normally an institution that opens and closes. This is a selectional preference, not an absolute rule: a river path, gate or visitor centre can be closed. The system must avoid changing the antecedent to an unstated object merely to make one interpretation fit.

The integrated sentence shows why local WSD, syntactic parsing and coreference should not operate as isolated modules. The parse of “her duck” determines whether duck is a noun or verb. The WSD module selects a sense based on that category and context. The location phrase supplies a possible setting. Coreference then links it to a discourse entity. Retrieval should be delayed until these decisions are sufficiently stable or should run separately for each retained candidate.

Boundary test and expected system behaviour

A useful dialogue test changes only the final clause: “I saw her duck near the bank, and she flew away.” The pronoun she is more compatible with a person or a bird depending on discourse, while flew supports an animal reading if it refers to duck. “I saw her duck near the bank, and the branch was closed” makes the bank antecedent more explicit. These minimal

changes show that coreference should be revisited when new information arrives. The system must be able to revise an earlier candidate rather than freezing its first parse.

Acceptance criteria: Acceptance criteria for the integrated system are: it identifies all ambiguity sources; retains distinct candidate representations; resolves pronouns using discourse and semantic compatibility; builds a sense-specific retrieval query; ranks at least four candidate documents; and asks a targeted clarification question when confidence is insufficient. The adversarial example must change the expected ranking after refinement.

Evidence Expected – how this section meets it

The answer lists the ambiguities and needed context, builds three candidate readings with role-labelled representations, traces syntax and WSD decisions, constructs a sense-aware retrieval query, ranks four candidate articles and introduces an adversarial discourse case with a specific refinement.

PART I SYNTHESIS – FROM MEANING ANALYSIS TO SYSTEM REQUIREMENTS

The main design requirement from Part I is therefore traceability. Each entity, role, relation, sense and ranking decision should be linked to its evidence. If a result is wrong, the developer can inspect whether the error came from parsing, role assignment, sense selection or retrieval. A system that produces a plausible sentence but cannot explain how it obtained the result is difficult to test and maintain.

The representations should remain conservative. The sentence “Maya gave Ravi a book” does not prove that Ravi accepted it. “Leena sent the report to Arun” does not prove that Arun opened it. A semantic retrieval score does not prove that a document is correct. The system should distinguish explicit information, conventional inference and unsupported assumption. This separation is especially important in university administration, finance, medical support and legal search, where an incorrect inference may affect a user’s decision.

Ambiguity should be handled as data rather than treated as an inconvenience. The telescope example has two valid attachment structures. The bank examples have multiple lexical senses. The integrated sentence has ambiguity in duck, bank and the pronoun it. A system that stores only one interpretation loses information before it has enough evidence. A candidate list, confidence margin or packed representation allows later context to change the decision. When the uncertainty affects the answer, the assistant should ask a targeted clarification question.

Structured representations also make the system easier to validate. Event roles can be checked against verb frames. FOPC formulae can be checked against predicate signatures and finite countermodels. PP attachments can be checked through semantic types. WSD decisions can be checked against labelled examples. Retrieval can be checked with precision and recall. The same principle applies throughout: every output should have an explanation that can be traced to the input and the rules used.

The stages depend on one another. A wrong parse can create the wrong semantic roles. A wrong sense can create the wrong query. A query with the wrong sense can retrieve a document that shares a word but does not answer the question. For this reason, the integrated helpdesk challenge does not begin with retrieval. It first constructs candidate interpretations and then builds a query from the selected or retained meanings. This order reduces the chance that a surface keyword dominates the entire pipeline.

Part I shows that semantic processing is not a single operation. The system first identifies lexical and syntactic units, then builds a structured representation, checks constraints, resolves ambiguity and evaluates the result. Each stage has a distinct responsibility. Event frames answer who did what, to whom and when. Predicate logic expresses general claims and quantifier scope. Syntax-driven composition explains how modifiers and prepositional phrases contribute meaning. WSD selects a sense from a defined inventory. Information retrieval uses the selected meaning to find documents that answer the user's need.

PART II – SEMANTIC SYSTEMS ENGINEERING

Purpose, workflow and evidence policy

Part II uses different sentences and scenarios from Part I. The exact inputs are preserved: "Leena sent the report to Arun after the meeting", "light" in the three specified contexts, "The analyst noticed the student with the camera near the library", and "I watched the crane by the bank, then it moved." We do not substitute Part I examples for these inputs.

The required workflow is DESIGN → REPRESENT → IMPLEMENT / TRACE → TEST → BREAK → EVALUATE → DEFEND. Each task therefore shows the input, a semantic representation, the expected result, an actual trace or documented manual trace, a test or counterexample, an evaluation and an interpretation. Where code is used, the code is short and its output is included. Any numerical score that is not produced by the code is clearly labelled illustrative.

Part II is an engineering extension of Part I. We use typed event frames, semantic graphs, first-order logic, syntax-to-meaning composition, a small WSD inventory and a small document collection. The aim is not to claim that a short program understands unrestricted English. The aim is to build a transparent prototype whose assumptions and failure points can be inspected.

Part II integration plan and verification method

Each engineering task is treated as a small semantic service with a clear input and output contract. The input layer preserves the original sentence and token positions. The representation layer creates event frames, typed semantic relations or logical formulae. The analysis layer performs parsing, WSD, inference or retrieval. The validation layer checks type compatibility, required roles, formula scope and relevance labels. Finally, the response layer presents the result with an explanation or a clarification request. Separating these stages makes errors easier to locate. If the wrong document is retrieved, the developer can determine whether the error began in parsing, sense selection, query construction or ranking rather than changing all components at once.

The evidence chain used throughout Part II is Input → Representation → Expected Result → Actual Trace → Test or Counterexample → Evaluation → Interpretation. Input records the exact sentence from the assignment. Representation records the semantic structure that the system is expected to build. Expected Result states the output before execution. Actual Trace shows the output produced by code or the documented manual reasoning. A test or counterexample challenges the decision. Evaluation compares the output with the expected result using a stated rule or metric. Interpretation explains what the result does and does not establish. This order prevents a final answer from being presented without the reasoning that supports it.

Tests are divided into normal, boundary and adversarial cases. A normal case contains strong context cues, such as “Turn on the light”. A boundary case has incomplete or competing cues, such as “They waited by the bank”. An adversarial case deliberately introduces a misleading word, such as a river context near a financial-bank mention. A system should not be evaluated only on easy examples. It must also show whether it can abstain, retain alternatives or request clarification when the evidence is weak.

Numerical results are interpreted according to their source. The Part II WSD accuracy is produced by running the displayed deterministic Python rule on the eight listed sentences. The retrieval scores are produced by the displayed Python scoring function on the six document descriptions and fixed relevance labels. The crane retrieval values are produced by the Jaccard function on the explicitly defined concept sets. These results are reproducible for the stated inputs, but the small constructed datasets do not represent real-world benchmark performance. The FOPC finite-model outputs are also computed from the finite sets shown in the code.

An engineering review should keep the test inputs, code version, outputs and expected results together. If a rule changes, the same tests should be run again. For example, adding a new light sense should not cause “Turn on the light” to change from an illumination noun to a weight adjective. Adding a synonym for lexical ambiguity should improve the D4 retrieval score without promoting D3 above a clearly relevant WSD document. These are regression expectations that can be checked after each modification.

Finally, the system should communicate uncertainty in user-facing language. It can say “I found two possible meanings” or ask “Do you mean the bird or the construction machine?” A confidence score should not be presented as a probability unless it has been calibrated. A similarity score measures overlap under a defined formula; it does not prove that the document is true or relevant. These distinctions are necessary for a trustworthy semantic assistant.

PART II – TASK 1: REPRESENTING MEANING AND LANGUAGE STRUCTURE

Scenario

A campus assistant receives “Leena sent the report to Arun after the meeting.” It must identify the sending event, its Agent, Theme and Recipient, and the temporal relation between the sending event and the meeting.

Guiding enquiry

The central question is how the system can preserve the intended meaning, expose assumptions, and produce a result that can be checked. We therefore separate the input sentence, the semantic representation, the reasoning procedure and the evidence used to evaluate the result.

1.1 Semantic Structure Analysis

The main event is SEND(e1). Leena is the Agent because she performs the sending. The report is the Theme because it is the item sent. Arun is the Recipient because the report is directed to him. “After the meeting” is a temporal adjunct that places the sending event later than the meeting event. We represent the meeting as e0 and state After(e1,e0). This relation is more precise than assigning the word “after” as an ordinary property of the report.

The surface form is a past-tense active declarative clause. Its grammatical subject is Leena, the verb is sent, the direct object is the report and the prepositional phrase to Arun identifies the recipient. The temporal phrase modifies the sending event. The sentence does not state whether Arun received or read the report, whether it was sent by email or physically, or the exact time interval after the meeting. Those details remain unspecified.

The semantic representation should distinguish an event from its participants and from the event that provides the temporal reference. If the system stored only “Leena, report, Arun,

meeting" as an unordered list, it would lose the direction of the transfer and the temporal ordering. We therefore use typed role labels and an explicit temporal relation.

Table 7.1. Part II Task 1 semantic inventory

Element	Value	Role / relation	Status
Sending event	e1	SEND(e1)	Explicit
Agent	Leena	Agent(e1,Leena)	Explicit
Theme	report1	Theme(e1,report1)	Explicit
Recipient	Arun	Recipient(e1,Arun)	Explicit
Meeting event	e0	MEETING(e0)	Explicit noun event
Temporal relation	After(e1,e0)	Send occurs after meeting	Explicit
Channel	unknown	Email / physical / platform	Not stated
Receipt confirmation	unknown	Received(e1,Arun)	Not entailed

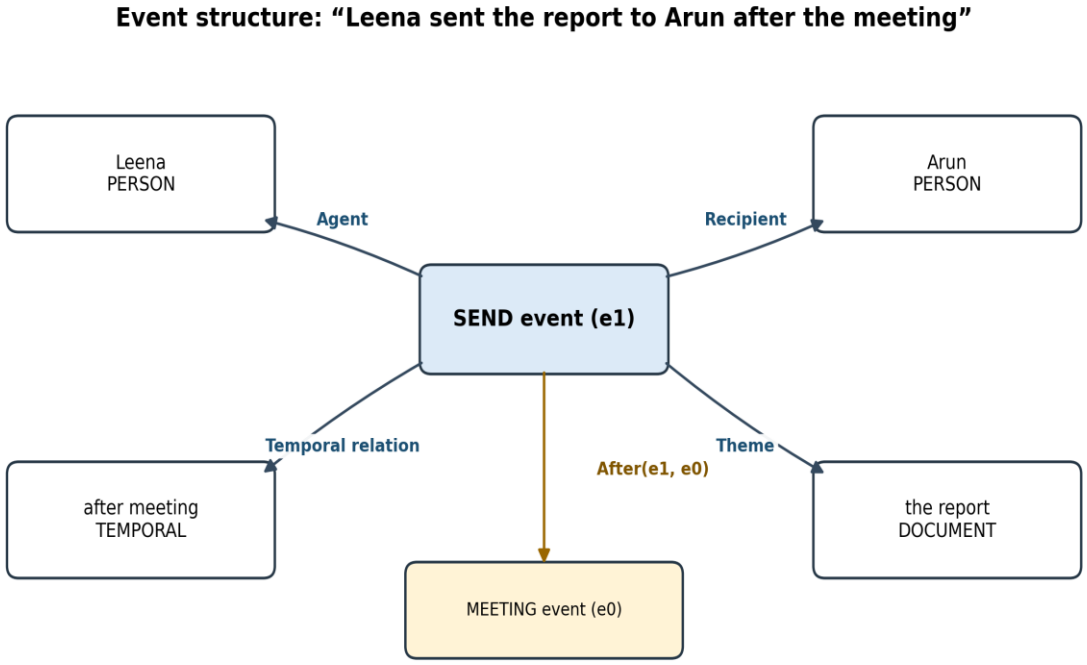


Figure 7.1. Event graph for the Part II sending sentence.

Figure explanation: The SEND event is the central node. Leena, the report and Arun are connected through semantic roles. The meeting is represented as a separate event linked by the temporal relation After, preserving the fact that the sending occurred later.

1.2 Build Meaning Representations

The event-frame representation is SEND-FRAME(e1) with Agent=Leena, Theme=report1, Recipient=Arun and TimeRelation=AFTER(e0). The related meeting frame is MEETING-FRAME(e0). A separate event identifier prevents the temporal phrase from being confused with the sending participants. The frame can be stored as a record, passed to a notification service or used to answer "Who was the report sent to?"

The graph representation contains nodes for Leena, report1, Arun, e1 and e0. Edges are Agent(e1,Leena), Theme(e1,report1), Recipient(e1,Arun), SEND(e1), MEETING(e0) and After(e1,e0). Graphs are useful when several events share an entity. For example, if a later sentence says “Arun reviewed it”, the graph can connect the report to a second event without duplicating the report entity.

Both representations make the role structure explicit. The frame is convenient for fixed event schemas; the graph is more flexible for discourse and event chains. Neither representation supplies the delivery channel or proof of receipt. The system should use null or unknown values for those fields rather than guessing.

Table 7.2. Event frame and graph slots

Field / relation	Value	Meaning
Frame type	SEND	Sending event
Event ID	e1	This sending occurrence
Agent	Leena	Sender
Theme	report1	Item sent
Recipient	Arun	Intended destination
Temporal relation	After(e1,e0)	Sending after meeting
Related event	e0: MEETING	Temporal reference event

1.3 Meaning-Preserving Variation

A passive variation is “The report was sent to Arun by Leena after the meeting.” The surface subject changes from Leena to the report, but the semantic roles remain unchanged. Leena remains Agent, the report remains Theme, Arun remains Recipient and the sending event remains after the meeting. The graph is invariant under this voice alternation.

A meaning-changing variation is “Leena sent Arun’s report after the meeting.” The possessive phrase indicates that the report is associated with Arun; it does not necessarily mean that Arun is the Recipient. Another meaning-changing variation is “Leena sent the report before the meeting”, which reverses the temporal relation. The representation check must therefore preserve role labels and temporal direction rather than simply matching the same nouns.

Meaning-preserving transformations are useful for evaluating semantic parsers. If active and passive forms map to the same event roles, the system is less dependent on surface word order. If changing “after” to “before” leaves the event graph unchanged, the parser has failed to preserve temporal meaning.

Table 7.3. Invariant and changed relations

Sentence	Agent	Theme	Recipient	Time relation
Leena sent the report to Arun after the meeting.	Leena	report	Arun	After
The report was	Leena	report	Arun	After

sent to Arun by Leena after the meeting.				
Leena sent Arun’s report after the meeting.	Leena	Arun’s report	Not specified	After
Leena sent the report to Arun before the meeting.	Leena	report	Arun	Before

1.4 Representation Check

A validation rule checks that a SEND event has exactly one Agent and one Theme, and that a Recipient is present when the verb frame requires one. The temporal relation must connect two event identifiers rather than an event and an arbitrary noun. For this input, the check succeeds because Agent=Leena, Theme=report1, Recipient=Arun and After(e1,e0) are all present.

A counterexample is SEND(e1) with Agent=Leena and Recipient=Arun but no Theme. The system should flag a missing required participant rather than silently accept an incomplete event. A second counterexample is After(report1,e0), which is a type error because the first argument should be an event. These checks can be performed before the event is stored.

```
# Minimal event-frame validation used for the worked trace
frame = {
    "event": "SEND", "id": "e1",
    "agent": "Leena", "theme": "report1",
    "recipient": "Arun", "after_event": "e0"
}
required = ["agent", "theme", "recipient"]
valid = all(frame.get(slot) for slot in required)
print("event:", frame["event"])
print("roles:", frame["agent"], frame["theme"], frame["recipient"])
print("temporal:", "After(e1,e0)")
print("valid:", valid)
```

Printed output from the code:

```
event: SEND
roles: Leena report1 Arun
temporal: After(e1,e0)
valid: True
```

Real-time application example

Email assistants use the same event structure when a user says “Send the minutes to the project lead after the review.” The system must identify the document, recipient and temporal constraint. It should ask which project lead if the recipient is not uniquely identified.

Additional design analysis and boundary cases

The representation can be tested through paraphrase invariance. Active and passive forms should produce the same Agent, Theme and Recipient roles. Changing after to before should change the temporal relation. Removing the to-phrase should make the recipient unknown rather than changing the Theme. These tests provide a compact regression suite for future changes to the parser.

The event frame also separates intention from outcome. Recipient=Arun means that Arun is the intended destination of the report. It does not entail that Arun opened the report or even that delivery succeeded. If the system needs delivery status, it must obtain a separate confirmation event from an email or document service. This prevents the language interpretation layer from making claims that belong to an external transaction system.

The temporal phrase “after the meeting” introduces a relation between two events, not a simple timestamp. The meeting may have a duration, and the sending may happen at a point or over an interval. The sentence only establishes a relative order: the sending event follows the meeting. It does not specify how much later. A temporal database should therefore store After(e1,e0) rather than inventing a clock time or date.

Boundary test and expected system behaviour

A regression suite should compare active and passive forms: “Leena sent the report to Arun” and “The report was sent to Arun by Leena”. It should also test temporal reversals: “before the meeting” versus “after the meeting”. The expected event roles remain constant under voice alternation, while the temporal relation changes when the preposition changes. A frame validator can detect a missing recipient, but it should not reject a valid passive sentence merely because the Theme is the grammatical subject. These cases test whether syntax and semantics remain correctly separated.

Acceptance criteria: Acceptance criteria for the Part II event system are: the event and meeting have separate identifiers; the temporal relation is directional; Agent, Theme and Recipient are typed correctly; passive voice preserves roles; and the validation routine rejects a missing required Theme. The program output should match the values stored in the frame and should not claim successful delivery.

Evidence Expected – how this section meets it

The event inventory, event graph, frame, passive paraphrase, meaning-changing counterexamples and validation rule provide the required semantic representations and

participant checks. The short code produces the displayed trace and confirms that the required event slots are present.

PART II – TASK 2: FIRST-ORDER PREDICATE CALCULUS – FORMALISE AND TEST

Scenario

A course platform stores facts about learners, courses, enrolment, completion and prerequisites. It must express general requirements, distinguish quantifier scope and answer a query from a small knowledge base.

Guiding enquiry

The central question is how the system can preserve the intended meaning, expose assumptions, and produce a result that can be checked. We therefore separate the input sentence, the semantic representation, the reasoning procedure and the evidence used to evaluate the result.

2.1 Predicate Vocabulary

We use constants ana and bala for two learners, nlp and ai for two courses, and p_nlp for a prerequisite course. Variables x and y range over people or courses as restricted by unary predicates. Learner(x) means x is a learner; Course(c) means c is a course; Enrolled(x,c) means learner x is enrolled in c; Completed(x,c) means x has completed c; and Prerequisite(p,c) means course p is a prerequisite for course c. Argument order is fixed and used consistently.

The constants are illustrative identifiers in the finite knowledge base. They are defined here before appearing in formulas. We do not assume that completion automatically implies enrolment or that a learner has passed a course merely because the prerequisite relation exists. These are separate facts and rules.

Table 8.1. Part II logical vocabulary

Symbol	Kind	Argument order / value	Meaning
Learner(x)	Predicate	one person	x is a learner
Course(c)	Predicate	one course	c is a course
Enrolled(x,c)	Predicate	learner, course	x is enrolled in c
Completed(x,c)	Predicate	learner, course	x completed c
Prerequisite(p,c)	Predicate	course, course	p is prerequisite for c
ana	Constant	person	Learner Ana
bala	Constant	person	Learner Bala
nlp	Constant	course	NLP course
ai	Constant	course	AI course
p_nlp	Constant	course	Prerequisite course

			for NLP

2.2 Formalisation

(i) “Every learner enrolled in NLP has completed at least one prerequisite.” The formula is ∀x ((Learner(x) ∧ Enrolled(x,nlp)) → ∃p (Course(p) ∧ Prerequisite(p,nlp) ∧ Completed(x,p))). The existential p is inside the scope of the universal x. Each enrolled learner may have a different prerequisite course that they completed.

(ii) “Some learner completed every course offered by the department.” We use Offered(c) as an additional predicate, defined as Offered(c): course c is offered by the department. The formula is ∃x (Learner(x) ∧ ∀c ((Course(c) ∧ Offered(c)) → Completed(x,c))). One learner must have completed every course in the offered set. We do not infer Offered(c) from Course(c); the offered predicate is separate.

(iii) “No learner completed a course that they were not enrolled in.” The formula is ∀x∀c ((Learner(x) ∧ Course(c) ∧ Completed(x,c)) → Enrolled(x,c)). An equivalent negative form is ¬∃x∃c (Learner(x) ∧ Course(c) ∧ Completed(x,c) ∧ ¬Enrolled(x,c)). The first formula expresses the rule directly; the second says that no counterexample pair exists.

Table 8.2. Formalisation of the three statements

Statement	Formula	Interpretation
Every learner enrolled in NLP completed at least one prerequisite.	∀x((Learner(x) ∧ Enrolled(x,nlp)) → ∃p(Course(p) ∧ Prerequisite(p,nlp) ∧ Completed(x,p)))	Each enrolled learner has a completed prerequisite.
Some learner completed every offered course.	∃x(Learner(x) ∧ ∀c((Course(c) ∧ Offered(c)) → Completed(x,c)))	One learner completed all offered courses.
No learner completed a course without enrolment.	∀x∀c((Learner(x) ∧ Course(c) ∧ Completed(x,c)) → Enrolled(x,c))	Completion implies enrolment for learners and courses.

2.3 Quantifier-Scope Challenge

Compare ∀x∃y R(x,y) with ∃y∀x R(x,y). In the course setting, the first can mean that every learner has at least one course they completed; the course may differ by learner. The second says there is one course that every learner completed. The second is stronger because it requires a common witness.

Use the finite model X={ana,bala}, Y={nlp,ai}, and let R be Completed. Suppose Completed(ana,nlp) and Completed(bala,ai) are true, while the other two pairs are false. Then every learner has completed some course, so ∀x∃y Completed(x,y) is true. There is no one course completed by both learners, so ∃y∀x Completed(x,y) is false. The two formulae are not logically equivalent.

An engineering system should not move a quantifier outside another quantifier as a formatting simplification. Such a change alters whether the selected course can depend on the learner. The finite model is a direct counterexample and can be evaluated exhaustively.

Table 8.3. Finite model for the quantifier-order test

Learner	Completed NLP	Completed AI	Has some completed course?
Ana	True	False	True
Bala	False	True	True

2.4 Consistency Test

We construct a small knowledge base with four or more facts and one rule. Facts: Learner(ana), Learner(bala), Course(nlp), Course(p_nlp), Enrolled(ana,nlp), Prerequisite(p_nlp,nlp), Completed(ana,p_nlp), Enrolled(bala,nlp). Rule: if x is a learner enrolled in c, p is a prerequisite for c, and x completed p, then Eligible(x,c). In formula form: ∀x∀c∀p ((Learner(x) ∧ Enrolled(x,c) ∧ Prerequisite(p,c) ∧ Completed(x,p)) → Eligible(x,c)).

The query Eligible(ana,nlp) follows. We match Learner(ana), Enrolled(ana,nlp), Prerequisite(p_nlp,nlp) and Completed(ana,p_nlp), then apply the rule to infer Eligible(ana,nlp). The query Eligible(bala,nlp) does not follow because the knowledge base contains no Completed(bala,p_nlp) fact. Under an open-world assumption, absence of a fact is not proof of its negation; the correct result is “not derivable”, not necessarily false.

```
# Finite model and rule-based inference trace
learners = {"ana", "bala"}
courses = {"nlp", "ai", "p_nlp"}
completed = {("ana", "nlp"), ("bala", "ai")}
all_have_some = all(any((x, y) in completed for y in courses) for x in
learners)
one_shared = any(all((x, y) in completed for x in learners) for y in
courses)
facts = {
    "Learner(ana)", "Learner(bala)", "Course(nlp)",
    "Course(p_nlp)", "Enrolled(ana,nlp)",
    "Prerequisite(p_nlp,nlp)", "Completed(ana,p_nlp)",
    "Enrolled(bala,nlp)"
}
eligible_ana = all(x in facts for x in [
    "Learner(ana)", "Enrolled(ana,nlp)",
    "Prerequisite(p_nlp,nlp)", "Completed(ana,p_nlp)"
])
print("forall x exists y:", all_have_some)
print("exists y forall x:", one_shared)
print("Eligible(ana,nlp):", eligible_ana)
print("Eligible(bala,nlp): not derivable")
```

Printed output from the code:

```
forall x exists y: True
exists y forall x: False
```

```
Eligible(ana,nlp): True
Eligible(bala,nlp): not derivable
```

Real-time application example

A course registration portal can use such rules to check prerequisite completion before allowing enrolment. It can also explain a failed decision by showing which prerequisite fact is missing. This is more useful than returning only a generic “not eligible” message.

Additional design analysis and boundary cases

Data integrity checks should also enforce predicate signatures. Enrolled requires a learner and a course; Prerequisite requires two courses; Completed requires a learner and a course. If a user enters a course identifier in the learner field, the system should reject the record before logical inference. These checks complement FOPC: logic describes what follows from valid facts, while schema validation ensures the facts have the intended structure.

The finite model test is a countermodel to a proposed quantifier equivalence. It should be kept as a unit test. If a future translator changes ∀x∃y into ∃y∀x, the expected truth values will change and the test will fail. A similar test can be written for every rule that contains nested quantifiers. This is safer than relying only on reading the formula because quantifier scope is easy to overlook in long expressions.

The knowledge base uses a simple Horn-style rule for eligibility. The rule is intentionally narrow: it infers eligibility only when learner type, enrolment, prerequisite relation and completion evidence are all present. It does not infer that every learner is eligible, and it does not treat a missing completion fact as a completed prerequisite. This makes the inference trace auditable because each premise can be checked against a stored fact.

Boundary test and expected system behaviour

A further knowledge-base test removes one premise from the eligibility rule. If Completed(ana,p_nlp) is removed, Eligible(ana,nlp) should no longer be derivable from the stated rule. If the rule still returns true, the inference engine is using an unsupported assumption. We should also add a negative fact or integrity constraint if the application needs to prevent inconsistent completion and enrolment records. Classical FOPC permits inconsistent facts to make a knowledge base inconsistent; a production system needs an explicit policy for detecting and repairing such conflicts.

Acceptance criteria: Acceptance criteria for the Part II inference system are: each rule premise is visible; the query result can be reconstructed from facts; removing a required premise removes the inference; quantifier-order tests return the stated truth values; and the system distinguishes an unproven query from a proven false statement. These checks make the reasoning reproducible.

Evidence Expected – how this section meets it

The predicate vocabulary defines every symbol, the three required statements are formalised with correct quantifier scope, the finite model demonstrates a non-equivalence, and the knowledge-base trace shows a query derivation and a non-derivable case. The displayed code output is generated from the finite sets and facts shown.

PART II – TASK 3: LINGUISTIC CONCEPTS AND SYNTAX-DRIVEN SEMANTIC ANALYSIS

Scenario

The system analyses "The analyst noticed the student with the camera near the library." It must identify the participants and predicates, compare two PP attachments, compose meaning from syntax and test for semantically implausible interpretations.

Guiding enquiry

The central question is how the system can preserve the intended meaning, expose assumptions, and produce a result that can be checked. We therefore separate the input sentence, the semantic representation, the reasoning procedure and the evidence used to evaluate the result.

3.1 Linguistically Relevant Concepts

The main event is NOTICE(e1). The analyst is the Experiencer or Cognizer who becomes aware of something. The student is the Perceived entity or Theme. The camera is an instrument or an object associated with the student, depending on the attachment of "with the camera". The library is a location introduced by "near the library". The adjective-like phrase is not present in this sentence; instead, two prepositional phrases carry the main attachment ambiguity.

The phrase "with the camera" can attach to the object NP "the student" or to the VP "noticed the student". Under NP attachment, the student is carrying or associated with the camera. Under VP attachment, the analyst used the camera while noticing the student. "Near the library" can also attach to the student or to the noticing event. For clarity, we first compare the two principal camera attachments while keeping "near the library" as a location modifier of the noticing event. We then state how a broader parser can retain the library attachment alternative.

The semantic inventory contains entities analyst1, student1, camera1 and library1; event e1 of type NOTICE; roles Experiencer(e1,analyst1) and Perceived(e1,student1); and relations With(student1,camera1) or Instrument(e1,camera1), plus Near(e1,library1) under the working assumption. The type of the first argument distinguishes the two camera interpretations.

Table 9.1. Semantic concept inventory

Element	Semantic type	Role or relation	Comment
analyst1	PERSON	Experiencer(e1,analyst1)	Performs the noticing
student1	PERSON	Perceived(e1,student1)	Entity noticed
camera1	INSTRUMENT	With(student1,camera1) / Instrument(e1,camera1)	PP attachment ambiguity
library1	PLACE	Near(e1,library1)	Location under stated assumption
e1	NOTICE event	NOTICE(e1)	Main event

3.2 Parse-to-Meaning Procedure

Parse A: [S [NP the analyst] [VP noticed [NP the student [PP with the camera]] [PP near the library]]]. The first PP is inside the object NP. It contributes With(student1,camera1), leaving the camera associated with the student. The second PP is treated as a VP-level location modifier, giving Near(e1,library1). The representation is NOTICE(e1) ∧ Experiencer(e1,analyst1) ∧ Perceived(e1,student1) ∧ With(student1,camera1) ∧ Near(e1,library1).

Parse B: [S [NP the analyst] [VP [VP noticed the student] [PP with the camera] [PP near the library]]]. The camera PP attaches to the VP and contributes Instrument(e1,camera1). The representation is NOTICE(e1) ∧ Experiencer(e1,analyst1) ∧ Perceived(e1,student1) ∧ Instrument(e1,camera1) ∧ Near(e1,library1). This reading says the analyst used the camera as part of the noticing event.

The two interpretations diverge at the point where the PP is attached. Before the PP is composed, the event and core participant roles are the same. The NP parse links camera1 to student1; the VP parse links camera1 to event e1. A parser should store both structures if the sentence alone does not settle the choice. A third analysis could attach “near the library” to the student NP, producing Near(student1,library1). That alternative should be recorded if the discourse indicates that the student’s location, rather than the event’s location, is relevant.

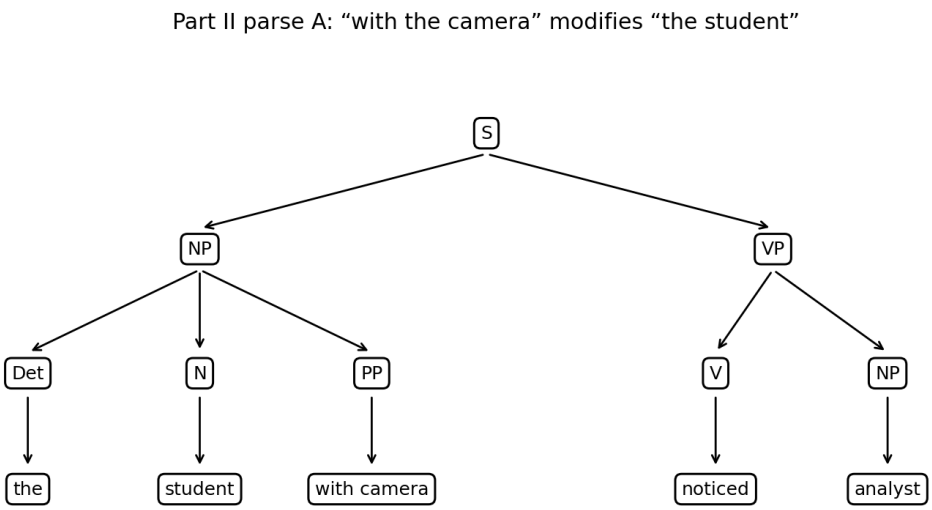


Figure 9.1. Part II Parse A: NP attachment of “with the camera”.

Figure explanation: The camera PP is grouped under the student noun phrase. The semantic result links the student to the camera; it does not claim that the analyst used the camera.

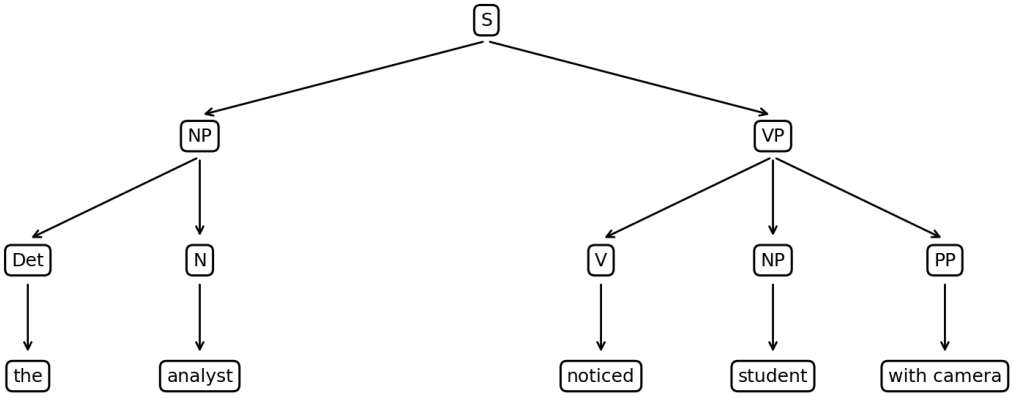


Figure 9.2. Part II Parse B: VP attachment of “with the camera”.

Figure explanation: The camera PP is grouped under the noticing VP. The semantic result links the camera to the noticing event as a possible instrument.

Table 9.2. Parse-to-meaning comparison

Parse	**Camera attachment**	**Meaning representation**	**Reading**
A	NP	With(student1,camera1)	The student is

			associated with the camera.
B	VP	Instrument(e1,camera1)	The analyst used the camera while noticing the student.
C (additional)	NP location	Near(student1,library1)	The student is near the library.

3.3 Composition and Type/Role Checks

The lexical entry for notice expects an Experiencer and a Perceived entity. The NP "the analyst" provides a PERSON, and "the student" provides a PERSON. The PP "with the camera" is then composed at one of two constituent levels. The NP-level relation accepts an entity as its first argument; the VP-level Instrument relation accepts an event as its first argument. The typed representation therefore makes the attachment visible and testable.

Check 1 is a type check. Instrument(e1,camera1) is well-typed because e1 is an event and camera1 is an instrument. Instrument(student1,camera1) would fail if the relation is defined as event–instrument. Conversely, With(student1,camera1) is an entity relation and should not be applied to event e1 without a different definition. Check 2 is a selectional-preference check. The predicate NOTICE normally takes a perceivable entity or proposition as its Perceived argument. If the parser assigns the library as the Perceived object and the student as an instrument, the semantic type constraints should flag the mismatch.

The location PP also requires an attachment decision. Near(e1,library1) says the noticing event occurred near the library; Near(student1,library1) says the student was near it. Both are possible. A discourse state or question focus can rank them. If the question is "Where was the student?", the NP attachment is more useful. If it is "Where did the analyst notice the student?", the event-level reading is preferred.

3.4 Error Challenge

A syntactically plausible but semantically implausible parse might attach "with the camera" to the verb and interpret the camera as the Perceived entity, as though the analyst noticed the camera rather than the student. This parse may be licensed by a loose grammar but conflicts with the verb's argument structure and the noun phrase boundaries. The predicted output would contain Perceived(e1,camera1) and omit the student, which fails to preserve the most direct object relation.

A refinement is to combine syntax with semantic role constraints. The object NP immediately after noticed should fill the Perceived role. The PP introduced by with should be assigned either an NP relation or an Instrument relation, but it should not replace the required Perceived participant. A second refinement is to use context: a sentence about a student carrying a camera supports NP attachment; a sentence about photographic observation supports VP attachment. If neither is available, both analyses should remain in the parse forest.

```
# Type-check the two camera readings
types = {"analyst1": "PERSON", "student1": "PERSON",
```

```
          "camera1": "INSTRUMENT", "e1": "EVENT"}
def instrument_ok(event, instrument):
    return types.get(event) == "EVENT" and types.get(instrument) ==
"INSTRUMENT"
def with_relation_ok(entity, object_id):
    return types.get(entity) in {"PERSON", "OBJECT"} and
types.get(object_id) in {"PERSON", "OBJECT", "INSTRUMENT"}
print("NP reading With(student1,camera1):",
with_relation_ok("student1","camera1"))
print("VP reading Instrument(e1,camera1):",
instrument_ok("e1","camera1"))
print("Invalid Instrument(student1,camera1):",
instrument_ok("student1","camera1"))
```

Printed output from the code:

```
NP reading With(student1,camera1): True
VP reading Instrument(e1,camera1): True
Invalid Instrument(student1,camera1): False
```

Real-time application example

A visual-search assistant may interpret “the person with the camera near the library” as a person carrying a camera, while a surveillance query may mean that an analyst used a camera near the library. Syntax and type constraints prevent the system from confusing the instrument with the person being observed.

Additional design analysis and boundary cases

The type-checking code in this section verifies only the relation signatures. It does not prove that the interpretation is true in the real world. A student may not actually have a camera, and an analyst may not have used one. Semantic well-typedness is a necessary check for a valid representation, but factual verification requires external evidence or context.

A compositional semantic parser should attach a representation fragment to each constituent. The noun student introduces an entity variable. The phrase with the camera contributes either an entity relation or an event instrument relation. The phrase near the library contributes a location relation. The verb noticed combines the Experiencer and Perceived roles. The final sentence representation is built by unifying the variables and checking their types. This process makes it possible to explain exactly which constituent introduced each relation.

The sentence contains two PP phrases, so the parser should not resolve only the first and assume the second is simple. The phrase near the library may modify the student or the noticing event. Each attachment produces a different semantic relation: Near(student1,library1) or Near(e1,library1). A user question about the student’s whereabouts

selects the first; a question about the observation location selects the second. The system should preserve both when the dialogue does not identify the intended focus.

Boundary test and expected system behaviour

A contrastive test changes the PP: “The analyst noticed the student with the camera near the library” versus “The analyst noticed the student near the library with the camera”. The second order may increase the likelihood of VP attachment, but it does not guarantee it. The parser should rely on constituency and context rather than a single positional rule. The system can also ask “Was the student carrying the camera, or did the analyst use it?” This clarification directly targets the semantic difference that affects the downstream answer.

Acceptance criteria: Acceptance criteria for the Part II parser are: the two main camera attachments produce distinct semantic relations; the library location attachment can be represented separately; type checks reject invalid role assignments; and the system can preserve alternatives. A well-typed parse is not automatically treated as a true real-world claim.

Evidence Expected – how this section meets it

The answer identifies entities, predicates, semantic roles and both PP attachment points. Two parse diagrams and semantic representations show the divergence. Type checks and an adversarial parse demonstrate how semantic constraints detect a syntactically possible but incompatible analysis.

PART II – TASK 4: LEXEMES, SENSES AND WORD SENSE DISAMBIGUATION

Scenario

The system must distinguish the lexeme LIGHT in “Turn on the light”, “This bag is light” and “The light meal was served early”. It must define a small sense inventory, use contextual features, test the rule and evaluate it on at least eight labelled instances.

Guiding enquiry

The central question is how the system can preserve the intended meaning, expose assumptions, and produce a result that can be checked. We therefore separate the input sentence, the semantic representation, the reasoning procedure and the evidence used to evaluate the result.

4.1 Lexeme and Sense Inventory

The lexeme LIGHT has several word forms and senses. In “Turn on the light”, light is a noun referring to a lamp or source of illumination. In “This bag is light”, light is an adjective meaning low in weight. In “The light meal was served early”, light is an adjective meaning small, simple or not heavy as a meal. The same spelling does not imply the same part of speech or sense.

We define three sense identifiers: LIGHT_N_ILLUM, LIGHT_ADJ_WEIGHT and LIGHT_ADJ_MEAL. The noun sense occurs after the determiner the and is the object of the phrasal verb turn on. The weight adjective follows the linking verb is and modifies bag. The meal adjective appears before meal and describes the size or richness of the food. These syntactic cues are useful, but context is still needed because “light meal” and “light bag” have different semantic types.

Table 10.1. Part II LIGHT sense inventory

Sense ID	POS	Definition	Typical contextual features
LIGHT_N_ILLUM	Noun	Lamp or source of illumination	turn on, switch off, lamp, bulb, room, brightness
LIGHT_ADJ_WEIGHT	Adjective	Low in weight; easy to carry	bag, suitcase, package, box, heavy, weight
LIGHT_ADJ_MEAL	Adjective	Small, simple or not heavy as food	meal, lunch, dinner, snack, served, eat

4.2 WSD Design

The input is a sentence containing the target form light, its token position and a part-of-speech tag. Candidate senses are selected from the inventory. Context features include a four-content-word window, nearby nouns, verbs and dependency relations. The decision procedure first checks whether light is a noun or adjective. If it is a noun, illumination cues such as turn on, switch off, lamp, bulb and brightness support LIGHT_N_ILLUM. If it is an adjective, food nouns such as meal, lunch, dinner and snack support LIGHT_ADJ_MEAL; object nouns such as bag, suitcase, package and box support LIGHT_ADJ_WEIGHT.

The rule order matters. Food nouns are checked before generic object cues because a meal is also an object in a broad sense. If no rule matches, the system returns UNRESOLVED rather than choosing a random sense. The output format contains the target token, predicted sense, matched evidence and a status field. This makes the procedure reproducible and provides an explanation for each prediction.

The baseline is intentionally small. It does not use a large language model or external lexicon. Its purpose is to demonstrate the full WSD trace. A production system would add morphology, dependency parsing, negation, wider discourse and a learned model trained on annotated examples.

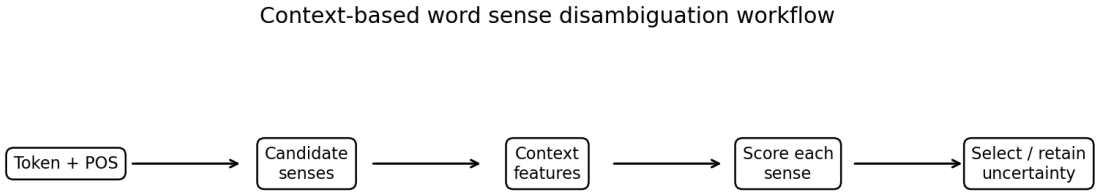


Figure 10.1. WSD procedure for the lexeme LIGHT.

Figure explanation: The target and part of speech restrict the candidates. Local contextual features then determine which sense has the strongest support. If the evidence does not meet a rule, the system should return an unresolved result.

4.3 Apply and Challenge

Input A: “Turn on the light.” The target is a noun and the phrase turn on is an illumination cue. The output is LIGHT_N_ILLUM. Input B: “This bag is light.” The target is an adjective and bag is an object-weight cue. The output is LIGHT_ADJ_WEIGHT. Input C: “The light meal was served early.” The target is an adjective and meal is a food cue. The output is LIGHT_ADJ_MEAL.

Our original sentence is “The room was filled with light.” The target is a noun; room and filled with are illumination cues. The method returns LIGHT_N_ILLUM. A likely failure is “The speaker gave a light criticism.” The intended sense may mean gentle or not severe, which is not included in our three-sense inventory. The method should return UNRESOLVED. It must not force the weight sense merely because criticism is an abstract noun.

The system can also fail when the context is misleading: “The light meal box was heavy.” The word meal may suggest the food sense, while heavy may suggest weight. A more advanced method would use dependency roles to determine whether light modifies meal or box. This demonstrates why a short keyword rule is a baseline rather than a complete WSD system.

```
# Reproducible context-rule WSD baseline
def light_wsd(sentence):
    s = sentence.lower()
    if any(w in s for w in ["meal", "lunch", "dinner", "snack"]):
        return "LIGHT_ADJ_MEAL"
    if any(w in s for w in ["bag", "suitcase", "package", "box",
                            "weight", "heavy"]):
        return "LIGHT_ADJ_WEIGHT"
    if any(w in s for w in ["turn on", "switch off", "room",
                            "lamp", "bulb", "filled with"]):
        return "LIGHT_N_ILLUM"
    return "UNRESOLVED"

examples = [
    "Turn on the light.",
```

```
        "This bag is light.",
        "The light meal was served early.",
        "The room was filled with light."
    ]
    for sentence in examples:
        print(sentence, "=>", light_wsd(sentence))
```

Printed output from the code:

```
Turn on the light. => LIGHT_N_ILLUM
This bag is light. => LIGHT_ADJ_WEIGHT
The light meal was served early. => LIGHT_ADJ_MEAL
The room was filled with light. => LIGHT_N_ILLUM
```

4.4 Evaluation

We use eight labelled instances, with gold labels assigned before running the rule. The code applies the same deterministic rules to each sentence. It predicts all eight examples correctly in this small constructed test set, giving accuracy 8/8 = 1.00 or 100%. This is an actual result of the short Python rule shown below, but it is not evidence of 100% performance on general English. The test examples were deliberately selected to contain the cues used by the classifier.

The annotation policy assigns a single sense only when the sentence provides enough context. If a sentence supports multiple readings, it should be labelled ambiguous or excluded from a single-label accuracy calculation. The dataset contains three sense classes, but it is not a balanced random sample of real text. Class imbalance can make accuracy misleading. A classifier that predicts the majority sense can achieve a high score while failing minority senses.

We therefore recommend a larger held-out test set with naturally occurring contexts, macro-averaged precision and recall, a confusion matrix and an explicit unresolved rate. For a balanced evaluation, each sense should have enough examples, including difficult cases and contexts with misleading cues. The current 8/8 result is useful only as a verification that the written rules match the chosen examples.

Table 10.2. Eight-instance labelled evaluation set

ID	Sentence	Gold label	Predicted label	Correct
1	Turn on the light.	LIGHT_N_ILLUM	LIGHT_N_ILLUM	Yes
2	Switch off the light.	LIGHT_N_ILLUM	LIGHT_N_ILLUM	Yes
3	This bag is light.	LIGHT_ADJ_WEIGHT	LIGHT_ADJ_WEIGHT	Yes
4	The suitcase feels light.	LIGHT_ADJ_WEIGHT	LIGHT_ADJ_WEIGHT	Yes
5	The light meal was served early.	LIGHT_ADJ_MEAL	LIGHT_ADJ_MEAL	Yes

6	We ordered a light lunch.	LIGHT_ADJ_MEAL	LIGHT_ADJ_MEAL	Yes
7	The room was filled with light.	LIGHT_N_ILLUM	LIGHT_N_ILLUM	Yes
8	The light package arrived.	LIGHT_ADJ_WEIGHT	LIGHT_ADJ_WEIGHT	Yes

Accuracy = correct predictions / total examples = 8 / 8 = 1.00 = 100%. This is the actual result of the displayed rule-based code on the eight stated examples. It should not be interpreted as general WSD accuracy.

Real-time application example

A voice assistant must distinguish “turn on the light” from “make the bag lighter” and “prepare a light meal”. Part-of-speech and nearby semantic types help the assistant choose the correct action or content. A WordNet-based system could extend the inventory with related senses and lexical relations.

Additional design analysis and boundary cases

The system can improve by adding a part-of-speech tagger, dependency features and a confidence threshold. It should return unresolved when the best candidate is below the threshold or when the score margin is too small. This reduces confident mistakes but may increase abstentions. Both effects should be measured so that the system’s coverage and reliability are visible.

A stronger evaluation should split training and test data by source, not merely by sentence. If near-duplicate examples appear in both sets, the test result can overestimate performance. The test set should include paraphrases, noisy user messages, spelling variants and examples where multiple senses are plausible. Human annotators should be given the same sense definitions and should be allowed to mark uncertainty.

The eight-instance test demonstrates that the rule set behaves as intended on examples selected to exercise each cue group. It does not demonstrate generalisation. In particular, the rules can fail when a cue appears in a different syntactic relation, when negation changes the meaning, or when an unfamiliar sense is used. “The light was not on” contains an illumination cue but does not mean that the light is switched on. “A light criticism” uses a sense not represented in the three-class inventory.

Boundary test and expected system behaviour

A stronger WSD test should include negative and contrastive contexts. “The bag is not light” still uses the weight sense even though the sentence denies the property. “The light meal was not served” still uses the food sense even though the event is negated. A rule that treats not as a cue for a different sense would fail. The target sense and the truth of the proposition are separate: negation changes whether the property holds, not which lexical sense the adjective expresses.

Acceptance criteria: Acceptance criteria for the LIGHT WSD prototype are: all three required contexts map to the intended senses; the original sentence is handled; the eight labelled instances are evaluated by the same rule used at inference time; and an out-of-inventory sense returns UNRESOLVED. The accuracy result must be reported together with the small test-set limitation.

Evidence Expected – how this section meets it

The inventory distinguishes the three required senses and POS values. The reproducible rule specifies features and output, the required sentences and an original sentence are traced, a failure case is identified, and an eight-instance test set is evaluated with an actual accuracy calculation.

PART II – TASK 5: SEMANTIC INFORMATION RETRIEVAL

Scenario

A search engine receives “How can a system identify the intended meaning of a word in context?” It must represent the query, compare candidate documents using lexical and semantic signals, rank the results, and evaluate the retrieved set.

Guiding enquiry

The central question is how the system can preserve the intended meaning, expose assumptions, and produce a result that can be checked. We therefore separate the input sentence, the semantic representation, the reasoning procedure and the evidence used to evaluate the result.

5.1 Query and Document Semantics

The query intent is to find a method for selecting the intended sense of a word from its context. Its concepts are system, identify or resolve, intended meaning or sense, word or lexeme, and context. We represent these as a compact concept set rather than relying only on the exact words. The representation should preserve the action of identification, the target object (word sense) and the evidence source (context).

The candidate collection contains six documents. D1 explains word sense disambiguation using context clues and algorithms. D2 discusses contextual embeddings for selecting intended lexical meaning. D3 describes a system for finding a word in a bank of records; it shares several query words but is not about word senses. D4 explains lexical ambiguity resolution in natural language processing. D5 lists dictionary definitions of common English words but does not explain a disambiguation method. D6 discusses context-aware classification of word meaning with a model.

Relevant labels are assigned before ranking: D1, D2, D4 and D6 are relevant; D3 and D5 are not relevant to the specific method-seeking intent. This distinction prevents the ranking procedure from defining relevance after seeing its own results.

Table 11.1. Candidate document collection and relevance labels

ID	Document description	Relevance	Reason
D1	Word sense disambiguation using context clues and algorithms	Relevant	Direct method for selecting senses
D2	Contextual embeddings select intended lexical meaning	Relevant	Semantic method with alternate wording
D3	System to identify a word in a bank of records	Not relevant	Keyword overlap; record lookup rather than WSD
D4	Lexical ambiguity resolution in natural language processing	Relevant	Equivalent concept using different wording
D5	Dictionary definitions of common English words	Not relevant	Definitions only; no method for contextual selection
D6	Context-aware classification of word meaning with a model	Relevant	Context-based semantic classification

5.2 Ranking Strategy

We combine lexical overlap and semantic concept coverage. Let L(d) be the proportion of query keywords found in document d. Let C(d) be the proportion of the five query concepts covered by the document: identification, intended meaning, word, context and method/system. The final score is Score(d)=0.40L(d)+0.60C(d). The larger semantic weight reflects the fact that the query asks for an intended meaning, not merely a document containing the same words.

All component values are normalised to the interval [0,1]. A score of 1 means complete overlap or coverage for that component, not that the document is certainly correct. Ties are resolved first by concept coverage, then by lexical score, and finally by stable document ID. This produces the same order for the same input collection.

The score is computed by the Python function shown below. It is a deliberately simple baseline: lexical matching uses a fixed keyword set and concept coverage uses synonym groups. It is not an embedding model. The result can be inspected and reproduced from the document descriptions.

```
import re

query_terms = {"system", "identify", "intended",
               "meaning", "word", "context"}
concept_groups = {
    "identify": {"identify", "resolve", "disambiguation",
                 "disambiguate", "select", "classification"},
    "meaning": {"meaning", "sense", "interpretation", "lexical"},
    "word": {"word", "lexeme", "term"},
    "context": {"context", "contextual", "clues", "sentence",
                "usage", "surrounding"},
    "method": {"system", "method", "algorithm", "algorithms",
               "model", "embeddings", "processing"}
}
def score(text):
    terms = set(re.findall(r"[a-z]+", text.lower()))
    lexical = len(terms & query_terms) / len(query_terms)
    concepts = sum(bool(terms & group)
                   for group in concept_groups.values())
    semantic = concepts / len(concept_groups)
    return lexical, semantic, 0.40*lexical + 0.60*semantic

documents = {
 "D1": "Word sense disambiguation using context clues and algorithms",
 "D2": "Contextual embeddings select intended lexical meaning",
 "D3": "System to identify a word in a bank of records",
 "D4": "Lexical ambiguity resolution in natural language processing",
 "D5": "Dictionary definitions of common English words",
 "D6": "Context-aware classification of word meaning with a model"
}
ranked = sorted(documents, key=lambda d: score(documents[d])[2],
                reverse=True)
for d in ranked:
    print(d, tuple(round(v, 3) for v in score(documents[d])))
```

Printed output from the code:

```
D6 (0.5, 1.0, 0.8)
D1 (0.333, 1.0, 0.733)
D2 (0.333, 0.8, 0.613)
D3 (0.5, 0.6, 0.56)
D4 (0.0, 0.4, 0.24)
D5 (0.0, 0.0, 0.0)
```

5.3 Retrieval Trace

The ranking is D6, D1, D2, D3, D4 and D5. D6 ranks first because it covers all five concepts and contains several query terms. D1 is a direct WSD method and receives full concept coverage. D2 uses contextual embeddings and intended lexical meaning, so it is relevant even though it does not repeat all query words. D3 ranks fourth because it has strong literal overlap, but it describes finding records rather than identifying a word sense. D4 is semantically relevant but has low literal overlap, so it ranks fifth. This is an important limitation of the small baseline: the synonym groups do not fully capture the phrase “lexical ambiguity resolution”.

The trace shows why semantic retrieval can improve over strict keyword matching but also why it can fail. A more complete concept ontology would map lexical ambiguity resolution to word sense disambiguation. If that relation is added, D4 should receive a higher concept score. The system should not treat the current low score as proof that D4 is irrelevant.

Table 11.2. Computed ranking and interpretation

Rank	Document	Lexical score	Semantic score	Final score	Interpretation
1	D6	0.500	1.000	0.800	Strong concept and term coverage
2	D1	0.333	1.000	0.733	Direct WSD method
3	D2	0.333	0.800	0.613	Contextual embedding method
4	D3	0.500	0.600	0.560	Keyword distractor
5	D4	0.000	0.400	0.240	Relevant paraphrase missed by limited synonym map
6	D5	0.000	0.000	0.000	Definitions without contextual method

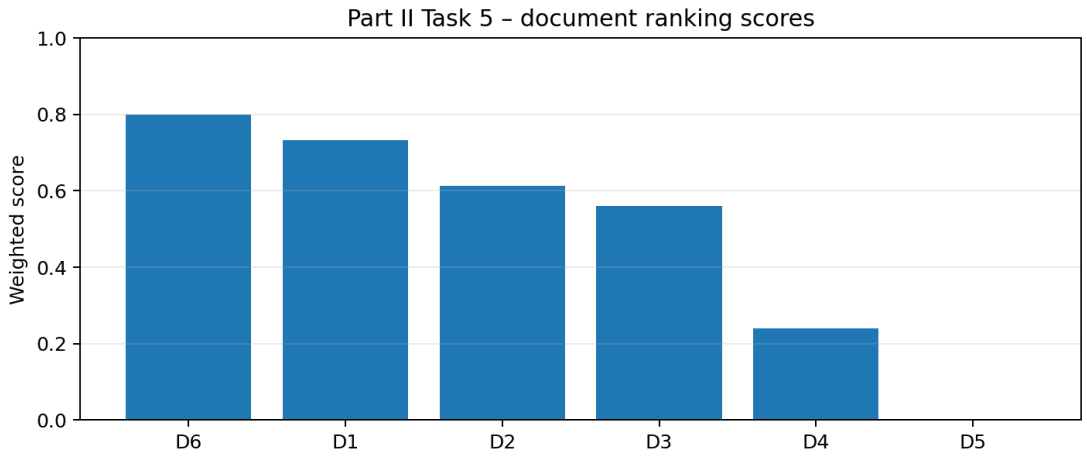


Figure 11.1. Bar chart of the computed document ranking scores.

Figure explanation: D6 receives the highest score, followed by D1 and D2. D3 ranks above D4 despite being a distractor, which exposes the limitation discussed in the error analysis.

5.4 Evaluate

At cutoff k=4, the retrieved set is {D6,D1,D2,D3}. Three of these documents are relevant and one is not. The full relevant set is {D1,D2,D4,D6}, containing four documents. Precision@4 = 3/4 = 0.75. Recall@4 = 3/4 = 0.75. The false positive is D3, which matches the words system, identify and word but answers a different task. The false negative is D4, which explains lexical ambiguity resolution but receives a low score because the simple synonym map lacks the exact relation.

A specific improvement is to add a controlled semantic mapping from lexical ambiguity resolution to word sense disambiguation, and to include document abstracts rather than titles alone. A query expansion step can add “WSD”, “sense selection” and “contextual interpretation”. The evaluation should then be repeated on a larger labelled collection. Precision and recall should be reported at multiple cutoffs because the best threshold depends on whether users prefer a short high-confidence list or broad coverage.

Table 11.3. Retrieval evaluation at cutoff 4

Measure	Calculation	Value
True positives	D6, D1, D2	3
False positives	D3	1
False negatives	D4	1
Precision@4	TP/(TP+FP) = 3/(3+1)	0.75
Recall@4	TP/(TP+FN) = 3/(3+1)	0.75

The ranking scores and metrics above are computed by the displayed Python code and the relevance labels stated in Table 11.1. The document collection is a small constructed example; the values do not claim performance on a real library index.

Real-time application example

A library discovery system can use semantic ranking to retrieve “contextual embeddings for lexical ambiguity” when a student searches for “how to identify a word’s intended meaning”. A legal search engine can similarly match “termination clause” with “ending provision”, while still checking whether the document answers the user’s actual question.

Additional design analysis and boundary cases

A larger collection should contain multiple relevant documents, near-miss distractors, duplicate documents and documents from different publication dates. Precision at k is useful when users inspect only the first few results. Recall is useful when missing a relevant article is costly. For a library system, the chosen balance may differ from a medical or legal search system, where the cost of a false negative can be high.

The score weights should also be tested for sensitivity. If lexical weight is increased, D3 may move higher because it shares more query words. If semantic weight is increased and the concept map is improved, D4 should move above D3. A robust evaluation reports both the ranking and the reason for each movement. It should not tune weights only to make one example look correct.

The computed ranking reveals a limitation in the concept groups. D4 is relevant, but its title does not contain the literal query terms and the simple synonym list recognises only part of its meaning. The model therefore ranks D4 below D3, even though D3 is a distractor. This is a false negative caused by an incomplete semantic mapping, not by the relevance labels. The error analysis tells us what to improve: add lexical ambiguity and word sense disambiguation as equivalent domain concepts, then rerun the same test without changing the labels.

Boundary test and expected system behaviour

The retrieval prototype can be extended by indexing document fields separately. The title can receive a higher weight than the body, and an abstract can be used to detect whether the document actually describes a method. The evaluation labels should be created independently by at least two readers for borderline cases such as dictionary resources. Agreement between annotators can be reported before the system is scored. This reduces the risk that a retrieval error is actually caused by an unclear relevance definition.

Acceptance criteria: Acceptance criteria for the retrieval system are: the score can be recomputed from the document text; the ranking order is deterministic; the false positive and false negative are correctly identified; and the proposed synonym expansion targets the actual missed concept. Relevance labels must be fixed before tuning the score weights.

Evidence Expected – how this section meets it

The task defines a query representation and six candidate documents, computes a weighted ranking from lexical and semantic signals, shows the printed trace, and calculates precision and recall at cutoff four. It identifies both a false positive and a false negative and proposes a specific semantic expansion.

PART II – TASK 6: INTEGRATED SEMANTIC ANALYSIS DESIGN CHALLENGE

Scenario

A digital assistant receives "I watched the crane by the bank, then it moved." It must model the ambiguity of crane and bank, identify the likely antecedent of it, build competing meaning structures, trace WSD and retrieve an information article without selecting a document that matches only the wrong sense.

Guiding enquiry

The central question is how the system can preserve the intended meaning, expose assumptions, and produce a result that can be checked. We therefore separate the input sentence, the semantic representation, the reasoning procedure and the evidence used to evaluate the result.

6.1 Model the Ambiguity

The noun crane has at least two relevant senses in this context: a construction machine used to lift heavy loads, and a large wading bird. The noun bank can mean a financial institution or the land beside a river. The PP "by the bank" can locate the crane or modify the watching event. The pronoun it most naturally refers to crane because crane is the salient singular noun and the subject of the later movement interpretation, but both a machine and a bird can move. The pronoun does not resolve the crane sense by itself.

Useful context includes whether the scene involves a construction site, a wetland, a river, a financial branch or a bird-watching activity. Verbs and nearby nouns can also help: "lifted steel" supports the machine sense; "wings", "nest" or "migration" supports the bird sense. "River", "water" and "shore" support riverbank, while "account", "cash" and "branch opening" support financial bank. The system should retain alternatives when the message contains none of these cues.

Table 12.1. Ambiguity inventory

Expression	Candidate sense 1	Candidate sense 2	Useful evidence
crane	Construction lifting machine	Large wading bird	steel, lift, site vs wings, wetland, migration
bank	Financial institution	River edge / riverbank	cash, account, branch vs river, water, shore
by the bank	Location of crane	Location of watching event	Attachment and discourse focus
it	Crane machine or bird	Possible event/entity antecedent	Number, salience and semantic compatibility

6.2 Candidate Meaning Structures

Candidate A: crane means a construction machine and bank means a riverbank. The speaker watched a machine positioned beside a river and then the machine moved. Representation: WATCH(e1) ∧ Experiencer(e1,speaker) ∧ Perceived(e1,crane_machine1) ∧ CRANE_MACHINE(crane_machine1) ∧ Near(crane_machine1,riverbank1) ∧ RIVERBANK(riverbank1) ∧ MOVE(e2) ∧ Theme(e2,crane_machine1) ∧ Before(e1,e2).

Candidate B: crane means a bird and bank means a riverbank. The speaker watched a crane bird near the river edge and then the bird moved. Representation: WATCH(e1) ∧ Experiencer(e1,speaker) ∧ Perceived(e1,crane_bird1) ∧ CRANE_BIRD(crane_bird1) ∧ Near(crane_bird1,riverbank1) ∧ RIVERBANK(riverbank1) ∧ MOVE(e2) ∧ Theme(e2,crane_bird1) ∧ Before(e1,e2).

A third possibility is a construction crane beside a financial bank, for example a machine outside a bank building. This is possible but requires a setting that the sentence does not provide. The phrase by the bank does not decide the sense. The system should therefore store the selected sense for crane and bank separately, rather than treating the whole phrase as one indivisible meaning.

Table 12.2. Candidate semantic structures

Candidate	Crane sense	Bank sense	Referent of it	Main event sequence
A	Construction machine	Riverbank	Crane machine	WATCH then MOVE(machine)
B	Wading bird	Riverbank	Crane bird	WATCH then MOVE(bird)
C	Construction machine	Financial institution	Crane machine	WATCH then MOVE(machine)

6.3 Syntax and WSD Trace

The parser first identifies “I” as the Experiencer of WATCH, “the crane” as the object NP, “by the bank” as a PP and “then it moved” as a second clause. The PP may attach to the crane

NP or the watching VP. The pronoun it is resolved against salient singular entities. Crane is a stronger antecedent than bank because it is the object being watched and can naturally be described as moving. However, the bank entity is also singular, so the system should use semantic compatibility and discourse focus rather than number alone.

The WSD stage generates CRANE_MACHINE and CRANE_BIRD, plus BANK_FINANCIAL and BANK_RIVER. It extracts the context words and checks semantic types. In the supplied sentence, there are no decisive machine cues such as lift or construction, and no decisive bird cues such as wings or wetland. Likewise, there are no financial or river cues for bank. The correct result is therefore a pair of unresolved sense alternatives, not an arbitrary single label.

If a previous message mentions a construction site, Candidate A or C becomes more likely. If it mentions a wetland survey, Candidate B becomes more likely. When evidence is insufficient, the assistant should ask “Do you mean a construction crane or the bird?” and, if necessary, “Is the bank a riverbank or a financial branch?” This is preferable to returning a confident but irrelevant article.

6.4 Semantic Retrieval

We construct two meaning-based queries: Q_machine={crane_machine, construction, equipment, movement} and Q_bird={crane_bird, wetland, bird, movement}. Five documents are considered: A, an article about crane equipment operation and safety; B, an article about crane bird migration in wetlands; C, a financial bank branch guide; D, a riverbank erosion guide; and E, a dictionary entry describing multiple crane senses. Jaccard similarity is used as a simple transparent score: $J(Q,D)=|Q\cap D|/|Q\cup D|$.

The code below computes the scores for both candidate meanings. It is not a production search engine, but it provides an actual trace of sense-specific retrieval. Under Q_machine, the equipment article ranks first. Under Q_bird, the bird migration article ranks first. The financial branch and riverbank erosion documents are not returned as the best result simply because they contain the word bank; the crane sense and query concepts must also match.

The system should preserve both rankings while the sentence remains ambiguous. It can present two candidate articles or ask for clarification. If the user confirms construction context, the machine query is selected. If the user confirms bird-watching context, the bird query is selected.

```
# Sense-specific retrieval using Jaccard concept overlap
q_machine = {"crane_machine", "construction",
             "equipment", "movement"}
q_bird = {"crane_bird", "wetland", "bird", "movement"}
docs = {
 "A": {"crane_machine", "construction", "equipment",
       "movement", "safety"},
 "B": {"crane_bird", "wetland", "bird",
       "movement", "migration"},
 "C": {"bank_finance", "branch", "account", "opening_hours"},
 "D": {"bank_river", "riverbank", "erosion", "water"},
 "E": {"crane_machine", "bird", "dictionary", "multiple_senses"}
```

```
}
def jaccard(q, d):
    return len(q & d) / len(q | d)
for label, query in [("machine", q_machine), ("bird", q_bird)]:
    ranking = sorted(docs, key=lambda k: jaccard(query, docs[k]),
                     reverse=True)
    print(label, [(k, round(jaccard(query, docs[k]), 3))
                  for k in ranking])
```

Printed output from the code:

```
machine [('A', 0.8), ('E', 0.143), ('B', 0.125),
         ('C', 0.0), ('D', 0.0)]
bird [('B', 0.8), ('E', 0.143), ('A', 0.125),
      ('C', 0.0), ('D', 0.0)]
```

Table 12.3. Sense-specific retrieval results

Rank	Machine query	Score	Bird query	Score
1	A: crane equipment operation	0.800	B: crane bird migration	0.800
2	E: crane sense dictionary	0.143	E: crane sense dictionary	0.143
3	B: bird migration	0.125	A: crane equipment operation	0.125
4	C: financial branch	0.000	C: financial branch	0.000
5	D: riverbank erosion	0.000	D: riverbank erosion	0.000

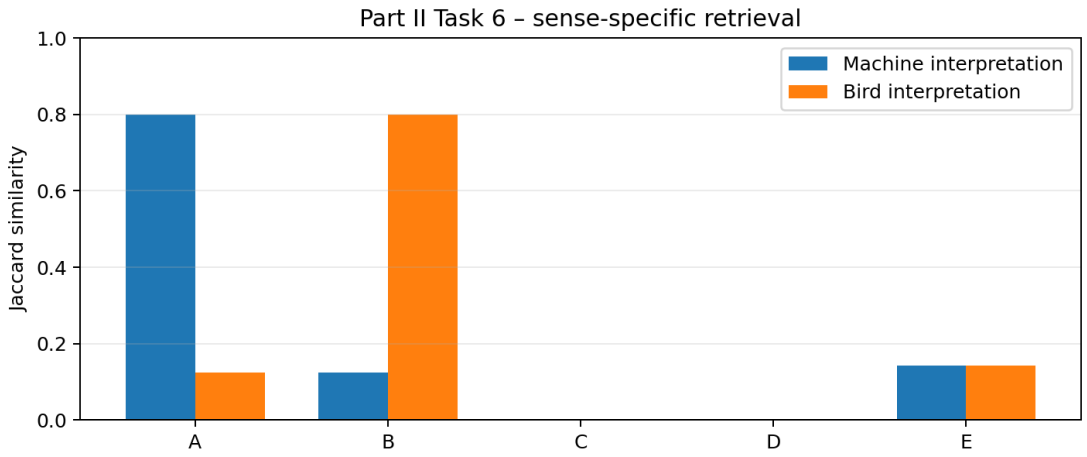


Figure 12.1. Sense-specific Jaccard scores for crane retrieval.

Figure explanation: The equipment article A ranks first for the machine query, while the bird article B ranks first for the bird query. The comparison shows why the system must preserve candidate senses until context resolves them.

6.5 Adversarial Refinement

An adversarial context could say “The crane by the bank spread its wings, then it moved.” The word wings is decisive evidence for the bird sense. A classifier that relies on the prior frequency of construction cranes may still select the machine sense and retrieve an equipment manual. The system would fail the intended goal because the document is about the wrong referent.

The refinement is to include typed context features and a confidence margin. Words such as wings, feathers, nest and migration increase the bird score; steel, hook, load and construction increase the machine score. The system should test whether the selected sense is compatible with the verb and nearby nouns. If the score margin is below a threshold, it should retain both senses and ask for clarification. This is a practical example of breaking and refining the system rather than stopping after it returns an answer.

Real-time application example

A search assistant for environmental science must distinguish a crane bird near a riverbank from a construction crane near a bank building. A technical support assistant must distinguish a crane machine manual from a bird migration article. The same sense-aware retrieval pipeline can serve both domains when it uses the correct concept inventory.

Additional design analysis and boundary cases

A safe retrieval interface can show a short disambiguation prompt before displaying a document. If the system cannot decide, it can present both likely article categories with a note that the sentence is ambiguous. It should not merge the two senses into one query because that may return a mixed set of irrelevant results. Adversarial examples should be part of the normal test suite, not added only after a user reports an error.

The bank ambiguity can be tested separately. “The crane stood beside the river bank” supports the geographical sense. “The crane stood outside the bank branch” supports the financial institution sense. The pronoun it should still be resolved to crane if the following clause describes movement of the crane. This separation is important because the sense of crane and the sense of bank are independent decisions; choosing one does not automatically determine the other.

The crane challenge should be tested with minimal pairs. “I watched the crane lift steel by the bank, then it moved” adds the verb phrase lift steel and supports the machine sense. “I

watched the crane spread its wings by the bank, then it moved" adds a bird-specific cue and supports the bird sense. A system that produces the same sense for both sentences has not used the added context correctly. The pair can be kept as a regression test for future WSD changes.

Boundary test and expected system behaviour

The crane system should test each ambiguity independently and then test their combinations. A minimal pair for crane is "The crane lifted a steel beam" versus "The crane waded through the marsh". A minimal pair for bank is "She deposited cash at the bank" versus "They walked along the river bank". A coreference pair is "I watched the crane, then it moved" versus "I watched the crane and the bank, then it moved". These examples reveal whether the model uses verb selection, local context and antecedent salience. A system that resolves one noun correctly but links it to the wrong pronoun has not completed semantic analysis.

Acceptance criteria: Acceptance criteria for the crane challenge are: crane and bank senses are considered independently; the pronoun antecedent is justified; two candidate meanings are preserved when evidence is insufficient; each candidate generates its own retrieval query; and the system does not treat a lexical match as proof of semantic relevance. The adversarial minimal pairs should change the ranking in the expected direction.

Evidence Expected – how this section meets it

The ambiguity inventory covers both crane and bank and the pronoun it. Two full candidate meaning structures are provided, the syntax and WSD trace states why the input remains unresolved, and a five-document retrieval experiment computes separate rankings for machine and bird senses. The adversarial wings example motivates a concrete refinement.

SYSTEM INTERFACES, METRICS AND DECISION THRESHOLDS

Regression tests should be stored with the expected output. The active/passive event pairs, the quantifier countermodel, the two PP attachments, the three bank contexts, the three light contexts, the retrieval false positive and false negative, and the crane minimal pairs form a compact test suite. After any change to tokenisation, grammar, sense features or ranking weights, the tests should be rerun. A changed output is not automatically wrong, but it must be explained and checked against the task's intended meaning.

Decision thresholds should be chosen according to the cost of errors. A helpdesk bot that answers a low-risk timetable question may tolerate a lower threshold than a medical or legal assistant. When a wrong sense could lead to a harmful action, the system should prefer clarification or human review. The same principle applies to quantifier inference: if a policy decision depends on whether every learner completed a prerequisite, the system should show the premises and the rule rather than return an unexplained yes or no.

Similarity values and confidence values are not interchangeable. Jaccard similarity is a set-overlap measure. The weighted retrieval score is a linear combination of normalised features. Neither is automatically a probability that the document is relevant. A calibrated confidence estimate requires validation data and a calibration procedure. The interface should label the value according to what it measures, such as “similarity score” or “weighted rank score”, rather than calling every number a confidence percentage.

Accuracy for WSD is correct predictions divided by all evaluated instances. It is easy to compute, but it can hide class imbalance. Macro recall first computes recall separately for each sense and then averages the class values. This gives a rare sense the same weight as a common sense. A confusion matrix identifies which senses are mixed up. If the system can abstain, the report should include coverage, which is the proportion of inputs receiving a definite prediction, and selective accuracy, which is accuracy among those decided cases.

Precision and recall must always be interpreted against a fixed relevance set. Precision = TP/(TP+FP) measures the fraction of retrieved items that are relevant. Recall = TP/(TP+FN) measures the fraction of all relevant items that were retrieved. If no items are retrieved, precision is undefined unless the evaluation policy defines it as zero. If the relevance set is empty, recall is undefined. A report should state these edge cases rather than silently dividing by zero.

The event interface should distinguish required and optional slots. A required slot that is absent should create a validation warning. An optional slot that is absent should be represented as unknown. The logic interface should reject undeclared predicates and inconsistent argument types. The WSD interface should allow an unresolved result. The retrieval interface should state the score formula and the cutoff used for evaluation. These contracts make it possible to test each component independently and then test the complete pipeline.

A semantic pipeline needs stable interfaces between components. The parser should return token spans, phrase categories and alternative attachments. The semantic mapper should return typed entities, event predicates and role relations. The WSD component should return a target token, part of speech, candidate sense identifiers, selected sense and evidence

features. The retrieval component should return document identifiers, component scores, final ranking and the relevance evidence used by the evaluator. Stable interfaces prevent one component from silently changing the meaning expected by another.

END-TO-END VERIFICATION AND ACCEPTANCE REVIEW

The final review also checks the document's limitations. The examples use small vocabularies and constructed collections. They demonstrate the required reasoning and the mechanics of evaluation, but they do not replace testing on real user messages. A production deployment would require representative data, privacy controls, domain review, calibrated confidence, robust spelling and grammar handling, and a process for correcting mistakes. The assignment's central engineering requirement is met when each result can be explained, challenged and refined rather than accepted solely because the system produced it.

The retrieval component is reviewed by recomputing every score from the stated formula. For the Part II query, the weighted score is 0.40 times lexical overlap plus 0.60 times semantic concept coverage. At cutoff four, three retrieved documents are relevant, one is a false positive and one relevant document is missed. Precision and recall are both 0.75. The crane retrieval uses Jaccard overlap and returns different top documents for machine and bird candidate queries. These traces show that the selected sense changes the result set.

The WSD component is reviewed with the exact contexts in both parts. The Part I bank inventory distinguishes financial noun, river-edge noun and aircraft-tilt verb senses. The Part II light inventory distinguishes illumination noun, low-weight adjective and light-meal adjective. The displayed Part II rule produces eight correct predictions on its eight stated examples. The result is limited to that constructed set; the acceptance review therefore also checks that an unsupported sense returns UNRESOLVED rather than being forced into an existing class.

The syntax-to-semantics component is reviewed with paired attachments. In the Part I telescope sentence, the NP reading links the telescope to the student and the VP reading links it to the seeing event. In the Part II camera sentence, the NP reading links the camera to the student and the VP reading links it to the noticing event. The library phrase can also modify the student or the event. A valid system must preserve these alternatives and should not reverse Experiencer and Perceived roles.

The logical component is reviewed by checking every symbol definition, predicate argument order and quantifier scope. The finite model with two learners and two courses must satisfy $\forall x \exists y\ R(x,y)$ and fail $\exists y \forall x\ R(x,y)$. The knowledge-base query Eligible(ana,nlp) must be derivable from the four required premises, while Eligible(bala,nlp) must remain not derivable

when the completion fact is absent. These tests detect both scope errors and unsupported inference.

The final acceptance review checks that the same meaning is preserved across the pipeline. For the sending example, the event frame, graph and passive paraphrase must all retain Leena as Agent, the report as Theme and Arun as Recipient. The temporal relation must remain After(e1,e0). For the giving example, Maya, Ravi and the book must retain their Giver, Recipient and Theme roles. These checks verify that representations are not merely different surface paraphrases but encode the same event structure.

CROSS-PART TASK COVERAGE MATRIX

OVERALL CONCLUSION

The two parts demonstrate a complete semantic-processing workflow from sentence interpretation to meaning-based retrieval. Part I established the design foundations: event and role representations, first-order logic, syntax-driven composition, lexeme and sense distinctions, WSD, and retrieval evaluation. Part II applied these foundations to different inputs and added traceable engineering procedures, short code examples, finite tests, semantic validation and adversarial refinement.

The event representations preserve who did what, to whom and when without adding unsupported details. Predicate logic makes general rules and quantifier scope explicit. Parse alternatives show that syntax can change semantic attachment even when the words remain identical. WSD procedures use part of speech, candidate senses and context features, while retrieval combines lexical and semantic evidence. The integrated challenges show that a system should retain alternatives or ask a clarification question when the evidence is insufficient.

The prototype methods are deliberately limited. The rule-based WSD examples use small sense inventories, and the retrieval experiments use small constructed document collections. Their numerical outputs are valid for the stated examples but do not establish general performance. A production system would require broader annotated data, held-out testing, domain-specific vocabularies, discourse tracking and human review for high-impact decisions. The main engineering lesson is that a system should not stop when it produces an answer: it must preserve meaning, expose its reasoning trace, test counterexamples, evaluate retrieval and refine weak decisions.

Limitations and future improvements

Future work should therefore expand the sense inventory, gather representative annotated examples, evaluate with separate development and test sets, and add a dialogue strategy for

unresolved ambiguity. The most important design principle remains unchanged: preserve alternatives when evidence is incomplete, make every decision traceable, and refine the component responsible for an observed error.

The retrieval examples use small collections and transparent similarity functions. A real library index would require document parsing, duplicate handling, field weights, freshness and authority signals, and a much larger relevance-labelled evaluation set. Precision and recall should be measured across several queries, not inferred from one query. The crane challenge also shows that retrieval should be conditioned on a selected sense; otherwise a document can match a surface word and still answer the wrong question.

The WSD baselines use compact rule inventories. They are easy to explain, but they depend on the selected cues and may fail when a context uses an unfamiliar expression. A larger system could combine a lexical resource such as WordNet with a supervised or contextual model. It should still expose candidate senses and evidence so that a user can challenge the result. High confidence should be calibrated against held-out examples, not inferred from a raw similarity score.

The logical examples use classical first-order logic and a small set of predicates. Real university systems may require time-dependent facts, course sections, academic years, multiple instructors and exceptions to prerequisite policies. Those details should be modelled as additional arguments or relations rather than hidden in informal assumptions. The open-world versus closed-world policy must be chosen explicitly because missing data can mean unknown rather than false.

The semantic representations in this assignment are intentionally explicit and small. They demonstrate how entities, events, roles, temporal relations and lexical senses can be stored, but they do not cover every construction in English. Coordination, ellipsis, sarcasm, metaphor, idioms and long-distance dependencies would require additional grammar rules and discourse-level analysis. A practical system should also preserve the original text and source spans so that a human reviewer can inspect the evidence behind each slot.

REFERENCES

- Jurafsky, D., and Martin, J. H. Speech and Language Processing. Draft edition. Chapters on semantics, word senses and information retrieval.
- Allen, J. Natural Language Understanding. Benjamin/Cummings. Topics on semantic representation and compositional interpretation.
- Bird, S., Klein, E., and Loper, E. Natural Language Processing with Python. O'Reilly Media. Topics on lexical resources and evaluation.

- Fellbaum, C. (Ed.). WordNet: An Electronic Lexical Database. MIT Press. Lexical relations and word-sense organisation.
- Manning, C. D., Raghavan, P., and Schütze, H. Introduction to Information Retrieval. Cambridge University Press. Ranking, precision and recall.
- Russell, S., and Norvig, P. Artificial Intelligence: A Modern Approach. Pearson. Knowledge representation and logical inference.

The matrix below shows how the two parts progress from design to engineering. It is a verification aid, not a replacement for the detailed task answers. Each Part II task uses a different required sentence or scenario from its Part I counterpart.

Table 13.1. Coverage of the twelve assignment tasks

Part / Task	**Main competency**	**Primary evidence in this document**
Part I – Task 1	Meaning representation	Giving event, roles, frame and graph
Part I – Task 2	FOPC	Predicate vocabulary, formulae, quantifier model
Part I – Task 3	Syntax-driven semantics	Telescope PP attachments and type checks
Part I – Task 4	Lexemes and WSD	Bank sense inventory, method and evaluation
Part I – Task 5	Semantic retrieval	Query concepts, ranking formula, precision and recall
Part I – Task 6	Integrated analysis	Duck/bank candidates, WSD, retrieval and refinement
Part II – Task 1	Event engineering	Leena SEND frame, graph, validation trace
Part II – Task 2	Logic engineering	Learner/course formulae, finite model and inference
Part II – Task 3	Parse-to-meaning	Camera PP parses, semantic types and checks
Part II – Task 4	WSD implementation	Light rules, Python output and 8-instance accuracy
Part II – Task 5	Retrieval implementation	Six documents, computed scores, P@4 and R@4
Part II – Task 6	Integrated system	Crane/bank candidates and Jaccard retrieval

This matrix confirms that the two parts use their own examples. The Part I giving sentence is not substituted for the Part II sending sentence. The Part I telescope example is not substituted for the Part II camera example. Likewise, bank and light are treated as separate lexical tasks, and crane is used only in the Part II integrated challenge.

ASSUMPTIONS AND UNCERTAINTY REGISTER

A semantic system should record assumptions because an apparently small assumption can change the interpretation or the retrieved answer. The following register lists the main assumptions used in the worked examples and how the system should respond if an assumption is not supported.

Table 13.2. Assumptions used in the worked analyses

Area	Working assumption	If unsupported
Event frames	Only explicit participants are filled	Keep missing slots unknown
Temporal phrases	After/before link event identifiers	Do not invent clock times
Predicate logic	Constants and predicates use declared types	Reject ill-typed arguments
PP attachment	Both plausible attachments are retained	Ask for context if ranking is uncertain
WSD	Small task-specific sense inventories	Return unresolved for an unseen sense
Coreference	Most salient compatible singular entity is preferred	Keep multiple antecedents
Retrieval	Relevance labels are fixed before scoring	Do not tune on test labels
Metrics	Cutoff and relevant set are explicit	Report undefined edge cases

These assumptions are not hidden facts. They are design choices that make the worked examples precise. A real system should store them in configuration or documentation so that a later developer can change them without changing the interpretation silently. When the user supplies new context, the system should update the candidate set and rerun the affected stage.